

AI for Economic Research: Dynamic Models, Language, and Agents

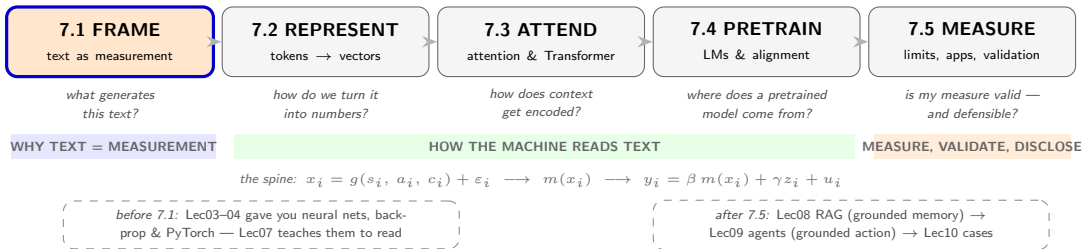
Lecture 7.1: Text as Economic Measurement

Zhigang Feng

2026-07-09 05:06:58

Lecture 7 in One Map

you are here ▼



Two case threads run through all five acts — **Thread A:** the EPU index (*Baker–Bloom–Davis 2016*), the transparent dictionary baseline every method must beat; **Thread B:** FOMC hawk–dove (*central-bank communication*), the running example for embeddings, attention, and validation. Watch for them.

Text as Data

Why language became an economic measurement problem

The Opening Hook: Reading Uncertainty out of Newspapers

In 2016, three economists asked: can you read policy uncertainty out of the newspaper?

- Baker, Bloom & Davis (2016, *QJE*): scan 10 major US newspapers, month by month, and count articles containing **all three** of {economic/economy}, **and** {uncertain/uncertainty}, **and** one of {Congress, deficit, Federal Reserve, legislation, regulation, White House}.
- Scale by each paper's total output, standardize, average across papers, index to 100:

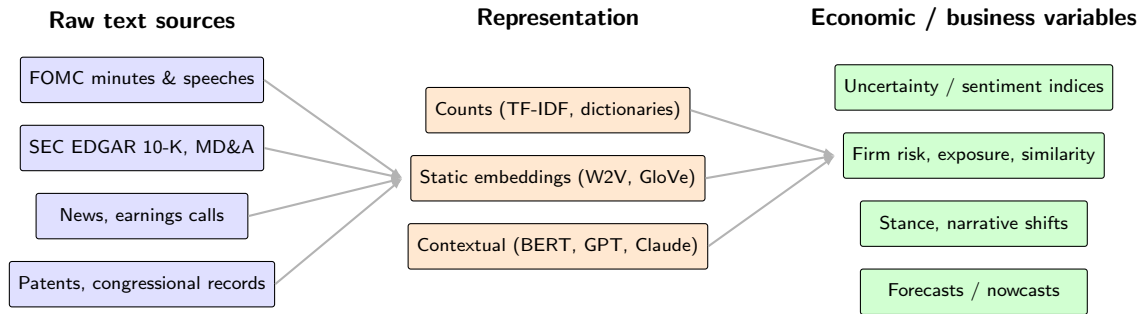
$$\text{EPU}_\tau = 100 \times \frac{u_\tau}{\bar{u}}, \quad u_\tau = \frac{1}{10} \sum_{p=1}^{10} \frac{c_{p\tau}/N_{p\tau}}{\sigma_p}.$$

where $p = 1, \dots, 10$ indexes newspapers, τ the month; $c_{p\tau}$ = qualifying articles in paper p ; $N_{p\tau}$ = paper p 's total articles (dividing removes size differences); σ_p = std. dev. of paper p 's scaled series (so no one paper dominates); u_τ = 10-paper average; $\bar{u}$ = its sample mean, so $\times 100$ sets the average to 100.

- No statistical agency publishes “policy uncertainty” — it is not directly observed anywhere. Yet in a VAR, EPU spikes precede investment and employment declines.

The move that matters: unstructured language became a *regression-ready variable* — this act asks what that takes, and where it goes wrong. (*Thread A begins.*)

Text as Economic Data — The Big Picture



The lecture builds the middle column rung by rung: Act 2 counts → static embeddings, Act 3 contextual machinery, Act 4 how those models are trained — and Act 5 turns the machinery into the right-hand column, with validation.

Why Economists Care About LLMs

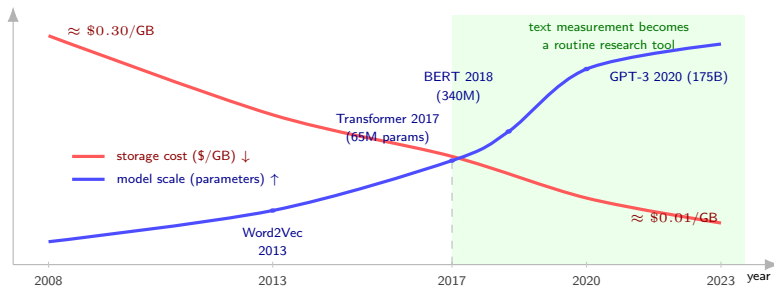
- **Text as data** — modern economic research increasingly leverages textual sources:
 - Corporate filings (10-K, 10-Q, MD&A)
 - Central bank speeches, FOMC minutes, ECB statements
 - Earnings calls, analyst reports, regulatory filings
 - News, social media, congressional records
- **Applications:**
 - Sentiment analysis for financial forecasting
 - Policy stance detection and uncertainty indices
 - New text-derived macroeconomic indicators
 - Summarization of long, technical documents
- **Why LLMs in particular:**
 - Capture nuanced language and domain-specific jargon
 - Handle long-range context (sentences, paragraphs, whole documents)
 - Provide a unified architecture for many tasks

Three Roles of Text in Economic Research

Role	What text is	Example
Measurement tool	A way to construct previously unobserved variables	Baker–Bloom–Davis (2016) Economic Policy Uncertainty index from newspapers
Behavioral outcome	A response variable revealing decisions or institutional change	Hansen, McMahon & Prat (2018): how FOMC transparency reform changed committee speech
Treatment variable	A quasi-experimental input affecting behavior	Central bank communication shocks and inflation expectations

Why Now? The Feasibility Window

Two curves crossed around 2017: the cost of handling text collapsed while model capability exploded.



Both quantities on log scales; curves are stylized interpolations through the marked real data points. Storage costs after McCallum's disk-price series; model sizes from Vaswani et al. (2017), Devlin et al. (2018), Brown et al. (2020).

The Spine

A data-generating process for text — and the two hazards it creates

An Identification-First View of Text

- **Engineering perspective first:** to a machine, a document is just a character string. But every string was *produced* — by a person, in a situation, for an audience. Before any econometrics, ask: what generated this string?
- **The data-generating process (DGP):**

$$x_i = g(s_i, a_i, c_i) + \varepsilon_i$$

- s_i : the latent **economic state** we want to measure (policy stance, sentiment, firm risk)
 - a_i : the speaker's **strategic framing** — audience management, spin, emphasis, tone chosen for effect
 - c_i : **institutional context** — venue, legal disclosure requirements, format conventions
- **Why this matters for a downstream regression** $y_i = \beta m(x_i) + \gamma z_i + u_i$: if the strategic framing a_i is correlated with the regression error u_i , then the text feature $m(x_i)$ is *endogenous* — the familiar omitted-variable problem, now hidden inside the language.

Two Firms, Same Risk, Different Words

The endogeneity hazard, made concrete. Imagine two firms facing the *same* objective political risk s_i .

- Firm A's management discloses risks proactively — building a reputation for transparency.
- Firm B's management avoids politically sensitive topics — afraid of spooking the market.
- Same s_i , different strategic framing $a_i \Rightarrow$ different texts $x_i \Rightarrow$ different measured risk $m(x_i)$.

- If the disclosure choice a_i correlates with anything in u_i (governance quality, investor base, financial health), $m(x_i)$ is **endogenous** — before any fancy NLP even enters.

Setting from Hassan, Hollander, van Lent & Tahoun (2019, QJE).

EPU: The Validation Standard (Thread A)

Why is a word-count index taken seriously? Because every step of it was audited.

- **Human audit:** a trained team hand-coded 12,009 articles (1900–2012) under a 65-page guide; the six policy terms were selected from $\sim 32,000$ candidate combinations to minimize false positives + false negatives.
- **Machine vs. human:** the “machine” is the automated word-count index itself — a program that flags every article matching the six-term rule and tallies the monthly counts, with no human judgment in the loop. Run over the same years the team hand-coded, it tracks the human-coded series at correlation 0.86 (quarterly, 1985–2012) and 0.93 (annual, 1900–2010); the gaps are uncorrelated with GDP growth.
- **Robustness:** dropping any one policy term barely moves the series; a 1,500-newspaper daily version correlates 0.85 with the 10-paper index.
- **And still — the identification critique:** does EPU *cause* downturns, or do downturns make journalists write “uncertainty”? The VAR ordering (EPU first) is a maintained assumption, not an identification argument.

Source: Baker, Bloom & Davis (2016, QJE).

Preview: The Machine Ahead

One equation will power everything in Acts 2–4

A Brief History: RNN → Transformer → GPT

- **2003:** Bengio et al. neural language model — continuous word vectors
- **2013:** Mikolov et al. *Word2Vec* — dense word embeddings at scale
- **2014:** Pennington et al. *GloVe* — global co-occurrence matrices
- **2017:** Vaswani et al. “*Attention Is All You Need*” — the Transformer replaces RNNs/LSTMs
- **2018:** Devlin et al. *BERT* (encoder); Radford et al. *GPT* (decoder)
- **2020–2022:** GPT-3, instruction tuning, RLHF → ChatGPT moment
- **2023–2026:** GPT-4/5, Claude (3–4), Gemini, LLaMA-3, DeepSeek; long-context (100k–1M tokens), open-weights ecosystem, agentic systems

Driver of the shift: attention scales better than recurrence, both in compute (parallel) and in quality (long-range dependencies).

The Autoregressive Principle

- An LLM is fundamentally a **next-token predictor**:

$$p(t^{(n+1)} \mid t^{(1)}, t^{(2)}, \dots, t^{(n)}).$$

- **Generation:** sample $t^{(n+1)}$ from this distribution, append to the context, and repeat.
- **What this single equation enables** (the full revisit comes at the end of Act 4, once pretraining and fine-tuning are on the table):
 - **Summarizing 10-K filings or FOMC minutes** — prompt with document + instruction; the model predicts summary tokens one by one
 - **Generating narrative forecasts** conditional on macro state — condition on data in the prompt; next tokens extend the narrative
 - **Producing scenario analyses in plain language** — the scenario is the context; prediction fills in the implications
 - **Writing and debugging code** for empirical work — same objective over a code corpus yields code completions
- **Key insight:** “everything is next-token prediction” is mathematically tiny but pedagogically and practically enormous.

AI for Economic Research: Dynamic Models, Language, and Agents

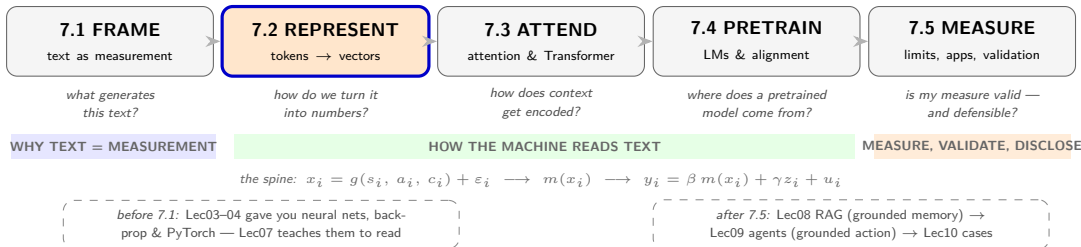
Lecture 7.2a: Preprocessing and Bag-of-Words

Zhigang Feng

2026-07-09 05:17:58

Lecture 7 in One Map

you are here ▼



Route: raw strings → clean text → tokens → counts (part 1/2 of REPRESENT — this deck builds the transparent count-based representations; part 2/2 turns counts into dense vectors).

From Strings to Tokens

Tokenization, subwords — and the Chinese special case

Tokenization: The First Step

- **Goal:** convert raw character sequences into discrete *tokens* that a neural network can consume.
- **Why not just split by spaces?**
 - Many economically relevant terms combine words, numbers, and symbols (e.g. S&P500, CO2, Q4_2025, stagflation)
 - Punctuation and formatting carry meaning ('\$1.2B' vs '1.2 billion')
 - Different languages have different word boundaries (or none at all)
- **What is a token?** A flexible unit:
 - A whole word ('inflation')
 - A subword ('infl', 'ation')
 - A single character or byte
 - A special marker (<EOS>, <PAD>, <USER>)

Vocabulary and the Tokenization Map

- Let A be the alphabet (Unicode characters). A tokenizer is a function:

$$\mathcal{K}: A^* \longrightarrow \{1, 2, \dots, M\}^*$$

mapping any text string into a sequence of **numbers** — each number is a positive integer that serves as an index into a lookup table of size M . The fact that the index is an integer is incidental; what matters is that it *uniquely identifies* each token, the same way a ZIP code uniquely identifies a location.

- **Why integers?** Neural networks need numeric inputs; each token ID will index into an embedding lookup table.
- **Practical constraint — context length:**
 - Models have a maximum input size n (typically 2,048 to 128,000 tokens; some 1M)
 - Long policy documents, FOMC transcripts, or 10-K filings often exceed n
 - Forces *chunking*, *summarization*, or *retrieval* (a hint at Lec08)
- Vocabulary sizes are typically 30,000–100,000 subword tokens. **To put this in perspective:** Merriam-Webster Unabridged lists $\approx 470,000$ entries; a college-educated native speaker actively uses roughly 20,000–40,000 words. A subword vocabulary of $M = 50,000$ therefore spans roughly 4/14

Illustrative Example: “GDP rose by 2%”

- A simple tokenization might produce

(GDP, rose, by, 2, %)

mapping to integer IDs, e.g. (541, 823, 71, 312, 49).

- **Why integer mapping matters:**

- Each ID becomes the index of a learned embedding vector
- Numeric tokens ('2', '3', '5') get *their own* embeddings

- **Injectivity matters for forecasting:** a well-designed tokenizer maps

- “inflation rate is 3.5%”
- “inflation rate is 3.6%”

to *different* token sequences. Otherwise small but economically meaningful differences would collapse into the same input.

- **Tokenizer/model coupling:** the tokenizer is part of the model. If you swap the tokenizer, all the embedding rows are meaningless — this is why every published LLM ships with its tokenizer.

Chinese Word Segmentation: Boundaries Must Be Inferred

English gets word boundaries for free; Chinese does not — so segmentation is itself a model.

- Chinese runs characters together; word boundaries must be *inferred*. Segmentation is a sequence-labeling problem: character by character, where does a word end?
- The classic ambiguity — 研究生命科学 splits as
 - 研究 | 生命科学 — “research on life science”, or
 - 研究生 | 命科学 — “graduate student” + a nonsense remainder.

Economic text has its own: 行业政策支持 parses as 行业政策 | 支持 (“industry-policy” + “support”) or 行业 | 政策支持 (“industry” + “policy-support”) — different head word, different meaning.

- **Tools:** Jieba, THULAC, LTP — each a dictionary plus a statistical/neural sequence labeler, trading speed against accuracy.
- **The economist’s trap — domain compounds:** terms like “monetary policy committee”, “macro-prudential framework”, “aggregate financing to the real economy” (社会融资规模) are not in general-purpose dictionaries; one mis-split changes the head word.
- **Fix:** load your domain glossary into the tool’s **user dictionary** so policy terms survive as whole tokens. Stopwords differ too: drop structural particles (的, 了, 是); keep logical connectives (因此 “therefore”, 然而 “however”) for information-extraction tasks.

Preprocessing Is a Modeling Decision

Every cleaning switch changes the downstream estimate

Tokenization Is a Preprocessing Choice

- Tokenization sits inside a larger **preprocessing pipeline**:

$$x^{\text{raw}} \xrightarrow{\text{clean}} x^{\text{clean}} \xrightarrow{\text{tokenize}} \{t_{ij}\} \xrightarrow{\text{represent}} z_i,$$

from raw scraped text $\rightarrow$ normalized text $\rightarrow$ integer token IDs $\rightarrow$ downstream numeric representation (TF-IDF, embedding, ...).

The Seven Preprocessing Choices

- **Lowercasing** — map Fed and fed to the same token. Compresses vocabulary, but erases proper-noun signal.
- **Stemming / lemmatization** — cut suffixes so *banking/banks/banked* collapse to *bank*. Can erase tense, plurality, policy-language nuance.
- **Stop-word removal** — drop high-frequency function words. Reduces noise; but negation (*not*) and modality (*may, will*) can matter for central-bank text.
- **Punctuation handling** — keep, strip, or tokenize separately. Percent signs, decimals, sentence boundaries all matter.
- **n -gram inclusion** — treat “*yield curve*” as one token rather than two. Captures multi-word economic concepts at the cost of a larger vocabulary.
- **Frequency cutoffs** — drop terms in $< k$ or $> K$ documents. Removes typos and boilerplate; can also remove rare-but-informative events.
- **Language filtering** — restrict to English or apply per-language pipelines. Crucial for multilingual corpora like ECB speeches.

One Token Boundary Can Flip the Sign

Where you draw token boundaries can reverse an economic conclusion.

FOMC-style sentence: “...growth is ***not sufficient*** to warrant further tightening.”

- **Strategy A (unigrams):** not and sufficient are separate features. If sufficient loads positively in training, a bag-of-words model can extract a *positive* signal from a negation.
 - **Strategy B (bigrams):** not_sufficient is one feature — the negation survives as a unit.
-
- Does the choice still matter under BERT-class models, whose attention can connect not to sufficient across positions? (Preview: mostly no — one reason Acts 3–4 exist.)
 - Either way: the tokenizer version and configuration belong in the replication package — they are part of the measurement instrument.

Bag-of-Words and TF-IDF

The transparent baseline: count, weight, audit

Bag-of-Words: The Document–Term Matrix

The simplest representation: count words, forget order.

- Toy corpus — doc 1: “central bank cuts rates to support growth”; doc 2: “central bank raises rates to curb inflation”.

	central	bank	cuts	raises	rates	support	curb	growth	inflation
doc 1	1	1	1	0	1	1	0	1	0
doc 2	1	1	0	1	1	0	1	0	1

- In general: D documents $\times$ vocabulary of size V (typically $V \gg 10^4$); entry $X_{d,v}$ is binary, a raw count, or a TF-IDF weight (two slides ahead).
- **The assumption you buy:** “central bank cuts rates to support growth” = “growth supports central bank rate cuts”. Harmless when counting topic words (EPU); fatal for direction, causation, negation.

Bag-of-Words: Two Built-in Consequences

Counting words and forgetting order leaves two statistical fingerprints on the matrix.

- **Sparsity:** a document uses only 1–2% of the vocabulary $\Rightarrow$ the matrix is $>99\%$ zeros; with $D \ll V$, OLS on raw counts is not even identified — dimension reduction (PCA, topic models) or regularization (Lasso) is mandatory.
- **Orthogonality:** every word is its own perpendicular axis. *Unemployment* $\perp$ *joblessness*: two articles about the same phenomenon in different vocabulary register as *unrelated*.

Orthogonality is the recall problem behind every dictionary method — and the motivation for the dense embeddings of T2b.

Term Frequency and Inverse Document Frequency

Weight words by how much they distinguish a document — not by how often they appear.

TF — share of document d that is word w :

$$\text{TF}(w, d) = \frac{\text{count}(w, d)}{\text{total words in } d}$$

“the Fed raised rates because inflation persisted” (7 words): every word gets $\text{TF} = 1/7 \approx 0.14$ — the scores as high as inflation.

Limitation: TF alone rewards frequency, not information. We need a counterweight.

IDF — rarity of w across N documents:

$$\text{IDF}(w) = \log\left(\frac{N}{\#\{\text{docs containing } w\}}\right)$$

Corpus of $N = 100$ FOMC speeches:

word	docs	IDF
<i>the</i>	100	0
<i>inflation</i>	60	0.51
<i>yield curve</i>	5	3.00

Rare $\Rightarrow$ informative $\Rightarrow$ up-weighted. *the* zeroes out automatically — no stop-word list needed.

TF-IDF — and When Counting Still Wins

Multiply term frequency by inverse document frequency; each document becomes a sparse vector in $\mathbb{R}^{|V|}$.

$$\text{TF-IDF}(w, d) = \text{TF}(w, d) \cdot \text{IDF}(w)$$

Orthogonality remains a real defect — yet counts win whole classes of research problems:

- **When the target is “how often do well-defined words appear”:** EPU (Thread A), the Loughran–McDonald finance dictionary, Hassan et al.’s bigram political risk — all count-based *by design*.
- **The killer feature is auditability:** you can list exactly which article and which phrase triggered the index — line-by-line external validation. No embedding offers that evidence chain.
- **Also the sensible default when:** the corpus is small ($D < 1,000$); compute is tight; or the measure must plug into a standard econometric pipeline with transparent features.

Choose the representation for the *role* the text plays (T1’s three roles): measurement wants transparency first; behavior and nuance want context — exactly where T2b and Act 3 head next.

Bridge: Counts Cannot See Synonyms

The bag of words has taken us as far as orthogonal axes can go.

- In the document–term matrix, *hawkish* and *tightening* are perpendicular; *unemployment* and *joblessness* share nothing. Every count-based measure inherits this blindness — the recall problem behind all dictionary methods.
- **Next deck:** learn coordinates *from co-occurrence*, so related words end up near each other — Word2Vec, GloVe, and the geometry that lets us do arithmetic with meaning.
- The DGP lens carries over unchanged: an embedding is just a richer $m(\cdot)$; the hazards a_i, c_i do not vanish because the vectors get denser.

Arc: T1 FRAME → T2 REPRESENT → T3 ATTEND → T4 PRETRAIN → T5 MEASURE.

AI for Economic Research: Dynamic Models, Language, and Agents

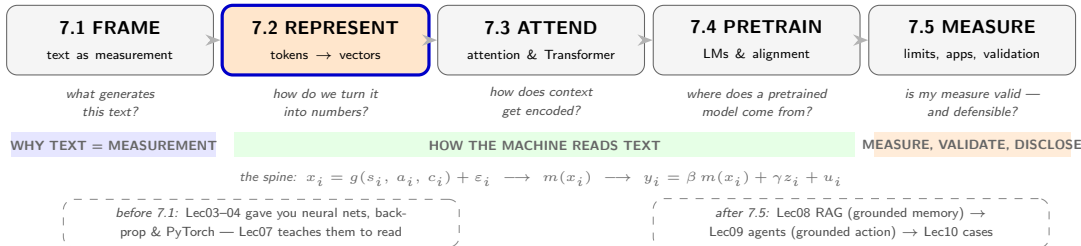
Lecture 7.2b: Word Embeddings

Zhigang Feng

2026-07-09 05:57:06

Lecture 7 in One Map

you are here ▼



Route: from counts to coordinates (part 2/2 of REPRESENT) — Word2Vec and GloVe, the geometry of meaning, what a direction in the space can measure, and where static vectors break.

From Counts to Coordinates

Learning a vector space in which meaning has geometry

Why Embed Tokens into Vectors?

- Neural networks operate on real-valued vectors, not discrete symbols.
- An **embedding** is a learned function

$$\mathcal{E}: \{1, \dots, M\} \longrightarrow \mathbb{R}^d,$$

mapping each token ID to a dense vector of dimension d (typically $d = 128$ to 4096).

- **What we want from embeddings:**
 - Semantically similar tokens should have nearby vectors
 - Algebraic operations (sums, differences) should reflect semantic relations
 - The mapping should be *learned* from data, not hand-designed
- **Bonus:** we will also need a *positional* signal so the model knows token order—more on this when we meet the Transformer (Act 3).

Three Approaches to Word Representation

Method	Type	Idea	Year
TF-IDF	Sparse, frequency	Re-weight word counts	1970s
Word2Vec	Dense, predictive	Predict neighbors in window	2013
GloVe	Dense, statistical	Factor co-occurrence matrix	2014
Transformer embeddings	Dense, contextual	Vary by surrounding tokens	2017+

Word2Vec: Dense Embeddings from Context

- **Distributional hypothesis** (Firth, 1957): “*You shall know a word by the company it keeps.*”
- **Idea** (Mikolov et al., 2013): for each word in a corpus, look at a window of $\pm L$ neighboring words. Learn a vector for the center word that *predicts* its neighbors (*Skip-gram*) or learn one that *is predicted by* its neighbors (*CBOW*).
- **Training output:** a single static vector $\mathbf{u}_w \in \mathbb{R}^d$ for each word w in the vocabulary, with d typically 100–300.
- **Geometric magic:** after training, simple vector arithmetic captures analogies, e.g.

$$\mathbf{u}_{\text{king}} - \mathbf{u}_{\text{man}} + \mathbf{u}_{\text{woman}} \approx \mathbf{u}_{\text{queen}}.$$

Words living on the same semantic axis differ by approximately the same vector.

Skip-Gram: The Idea in One Slide

Every word carries a vector; training nudges the vectors of words that appear together closer, and random pairs apart.

- The *score* of a (center word, nearby word) pair is just the dot product of their vectors, $\mathbf{u}_{\text{center}}^\top \mathbf{v}_{\text{context}}$ — large when the two point the same way. (Turning that score into a probability is a multinomial logit — familiar territory for economists.)
- Training sweeps the whole corpus and, over and over:
 - **pulls together** pairs that actually co-occur — push the score *up* (*inflation* near *persistent*);
 - **pushes apart** a handful of random pairs — push the score *down* (*inflation* vs. *basketball*).

This push-apart shortcut is called *negative sampling*: it avoids re-scoring the entire vocabulary at every step.

- **Result:** words that keep similar company end up with similar vectors — meaning emerges from nothing but “who appears near whom.”

Skip-Gram: Walkthrough on “central banks target inflation”

- **Sentence:** “*central banks target inflation*”, window radius $L = 2$, center word = *target*.
- Training pairs formed from this center word:

$(target, central), (target, banks), (target, inflation).$

- For each pair, the score is the dot product $\mathbf{u}_{target}^T \mathbf{v}_{context}$. Gradient descent *increases* this score for observed pairs and *decreases* it for random (negative) pairs.
- **Across the whole corpus:** *target* appears near *rates, inflation, policy, Fed*. So do *raise* and *cut*. Consequently

$\mathbf{u}_{target}$ ends up close to $\mathbf{u}_{raise}$, $\mathbf{u}_{cut}$ and far from $\mathbf{u}_{apple}$.

- **Takeaway:** analogies emerge because consistent co-occurrences place related words at similar positions in $\mathbb{R}^d$.

GloVe: Count Once, Then Fit

Word2Vec slides a window and predicts neighbors; GloVe takes a shortcut — count how often each pair of words co-occurs across the whole corpus, then fit vectors to reproduce those counts.

- Let X_{ij} = the number of times words i and j appear near each other in the entire corpus. (*Fed* and *rates* might co-occur 820 times; *Fed* and *apple*, just 3.)
- Pennington et al. (2014) ask each pair of vectors to reproduce its co-occurrence count:

$$\underbrace{\mathbf{u}_i^\top \mathbf{v}_j}_{\text{how aligned the vectors are}} + \underbrace{b_i + b_j}_{\text{each word's baseline}} \approx \underbrace{\log X_{ij}}_{\text{how often they co-occur}}$$

- **The whole model in one sentence:** *two words that keep each other's company should get vectors that point the same way.* Everything else is just fitting that sentence by least squares.

GloVe: Read It Like a Gravity Equation

An economist has met this equation before — it is the **gravity model of trade**, term for term.

$$\underbrace{\log(\text{trade}_{ij})}_{\text{how much } i, j \text{ trade}} \approx \text{FE}_i + \text{FE}_j + (\text{pair closeness}) \iff \log X_{ij} \approx b_i + b_j + \mathbf{u}_i^\top \mathbf{v}_j$$

- $\log X_{ij}$ — how much the two words “trade” with each other. We take logs because co-occurrence counts span orders of magnitude (3 vs. 820) — the same reason we use $\log$ GDP.
- b_i, b_j — **word fixed effects**. *the* co-occurs with everything simply because it is everywhere; the fixed effects soak up that baseline popularity so we don’t mistake it for meaning.
- $\mathbf{u}_i^\top \mathbf{v}_j$ — what is *left over* once popularity is removed: the pair’s **specific affinity**. This, and only this, becomes meaning.

The payoff: words that co-occur more than their popularity alone would predict get vectors pointing the same way; words that never meet stay at right angles. *Meaning becomes geometry, estimated like a regression.*

GloVe at Work: A Central-Bank Corpus

Watch “monetary-policy vocabulary” fall out of nothing but co-occurrence counts.

co-occurs with →	<i>rates</i>	<i>inflation</i>	<i>apple</i>
Fed	820	410	3
ECB	540	390	2
hawkish	210	160	0

What the fit does, row by row:

- *Fed* & *rates* co-occur 820 times ($\log X \approx 6.7$). The baselines $b_{\text{Fed}} + b_{\text{rates}}$ cannot explain a number that big — the only way to fit it is to make $\mathbf{u}_{\text{Fed}}$ and $\mathbf{v}_{\text{rates}}$ **point the same way**.
- *Fed* & *apple* co-occur just 3 times ($\log X \approx 1.1$) — about what the baselines predict for *any* two words. Nothing is left to explain, so their vectors end up **at right angles** (unrelated).
- *Fed*, *ECB*, and *hawkish* all keep the *same company* (rates and inflation, never apple), so all three get pulled into the **same corner** of the space — to the geometry they are near-synonyms.

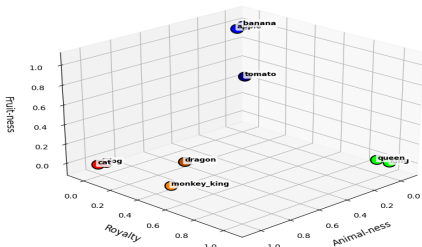
The punchline: nobody told the model what a central bank is. Rows with similar co-occurrence patterns are forced to similar coordinates, and a “monetary-policy” cluster emerges purely from counting.

The Geometry of Meaning

Distances, directions, analogies — and the biases baked in

A 3D Toy of Word Embeddings

Enhanced 3D Word Embeddings Visualization



The three axes loosely correspond to “Animal-ness”, “Royalty”, and “Fruit-ness”. Words like *cat*, *king*, and *apple* cluster along the dimension that fits them best. Real LLM embeddings live in hundreds of dimensions, but the same intuition holds.

We compare words by the *direction* of their vectors — **cosine similarity** $\cos(\mathbf{u}, \mathbf{v}) = \mathbf{u}^\top \mathbf{v} / (\|\mathbf{u}\| \|\mathbf{v}\|)$ — not raw distance: frequent words grow longer vectors in training, and direction strips that out.

Analogies and the Bias Problem

- **Geometric magic** (Mikolov et al., 2013): simple vector arithmetic captures analogies.

$$\mathbf{u}_{\text{king}} - \mathbf{u}_{\text{man}} + \mathbf{u}_{\text{woman}} \approx \mathbf{u}_{\text{queen}} \quad \mathbf{u}_{\text{Paris}} - \mathbf{u}_{\text{France}} + \mathbf{u}_{\text{Italy}} \approx \mathbf{u}_{\text{Rome}}$$

- Words on the same semantic axis differ by roughly the same vector — gender, capital city, monetary stance. (Caveat: the clean pattern weakens for specialized, low-frequency vocabulary — validate on your own terms.)
- **Bias is encoded too** (Bolukbasi et al., 2016, *NeurIPS*; Caliskan, Bryson & Narayanan, 2017, *Science*):
 - $\mathbf{u}_{\text{man}} - \mathbf{u}_{\text{woman}}$ has a large component on *programmer* vs. *homemaker*.
 - Word embeddings reproduce IAT-style biases at human magnitudes.
 - For economic text: occupation/gender, race/credit-risk, region/sentiment correlations all transfer from the training corpus into the geometry.
- **Lesson:** embeddings are a mirror of the corpus. Checking that a sentiment vector tracks *policy* stance and not *speaker identity* is a real research step (returns in T5b) — and the encoded bias can itself become the research object.

Four Generations of Representation, One Table

Every step toward “understanding” trades away interpretability.

	Counts / TF-IDF	Word2Vec	Contextual (BERT)
Captures meaning?	no (orthogonal)	yes (distributional)	strong (context-aware)
Same word, many senses?	no	one static vector	varies with context
Interpretable / auditable?	high (word = axis)	low (latent space)	very low
Typical econ use	EPU, dictionaries	culture / network scores	sentiment, stance

Rule: when validity rests on an auditable, line-by-line evidence chain, the fancier representation is often the *worse* choice. Pick the rung your research design needs — not the newest one.

From Analogies to Constructs: The Hawk–Dove Direction (Thread B)

Define an economic concept as a direction in the space.

- Candidate monetary-stance axis: $\hat{\mathbf{v}} = \mathbf{u}_{\text{hike}} - \mathbf{u}_{\text{cut}}$. Score any FOMC passage by the cosine of its (averaged) vector with $\hat{\mathbf{v}}$.
- The same trick, generalized: culture scores from seed-word directions (Li, Mai, Shen & Yan 2021); time-varying product-market networks from 10-K business-description similarity (Hoberg & Phillips 2016).
- **Validation idea:** a genuine stance direction should co-move with policy — check the $\hat{\mathbf{v}}$ -scores against subsequent fed-funds-rate changes before using them in a regression.

The separability assumption. *king* – *man* + *woman* works because the difference is (mostly) one clean dimension — everything else cancels. But *hike* co-occurs with overheating and *cut* with recessions, so $\hat{\mathbf{v}}$ may encode **stance and the business cycle at once** — like a bond that bundles duration and credit risk in one price. Purge or control for the cycle dimension before calling it “policy stance”; ask which institutional features make the coupling worse.

AI for Economic Research: Dynamic Models, Language, and Agents

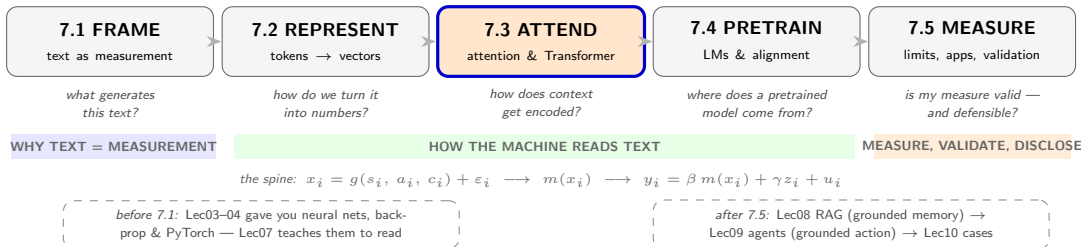
Lecture 7.3a: The Attention Mechanism

Zhigang Feng

2026-07-07 05:00:08

Lecture 7 in One Map

you are here ▼



Route: one attention head, end to end (part 1/2 of ATTEND) — Q/K/V projections → scaled scores → softmax → weighted blend → multi-head; part 2/2 wraps the head into the full Transformer layer.

Why Context Needs a Mechanism

From “one vector per word” to “one vector per occurrence”

How the Transformer Works in Plain Language

- Consider the sentence: *“The Fed raised rates to combat inflation.”*
- When you read the word **rates**, you instinctively ask: *“rates of what? raised by whom? why?”* — your eye jumps back to *Fed* (subject) and forward to *inflation* (motivation).
- **A Transformer does exactly this, for every token simultaneously:**
 1. Every token asks: *“which other tokens are most relevant to understanding me?”*
 2. Each token collects a weighted blend of information from every other token in the sequence
 3. All positions are processed in **parallel** — not one-by-one as in an RNN
- Repeat this “everyone looks at everyone” operation L times (e.g. $L = 96$ in GPT-3) with different learned weights, and the model builds deeply contextualized representations.
- **Engineering name for this operation:** scaled dot-product self-attention. The next slides unpack the five steps in detail.
- **Contrast with Word2Vec / GloVe.** Those models produced *one vector per word type*, independent of context — *bank* in “river bank” and *bank* in “central bank” share the same $\mathbf{u}$. A Transformer produces *one vector per token occurrence*, computed from its neighbors — the two *banks* end up at different points in $\mathbb{R}^d$. **Static** $\rightarrow$ **contextual** is the single conceptual jump.

The Transformer as a Function

- A Transformer is a function

$$\mathcal{T}: (\mathbb{R}^d)^n \longrightarrow (\mathbb{R}^d)^n, \quad [e^{(1)}, \dots, e^{(n)}] \mapsto [e'^{(1)}, \dots, e'^{(n)}]$$

mapping n input vectors to n output vectors of the same dimension. Inputs are token embeddings plus positional encodings (details in T3b); outputs are *contextualized* embeddings.

- **What “enriched” means.** Each $e'^{(i)} \in \mathbb{R}^d$ encodes at once (a) the identity of token $t^{(i)}$, (b) its grammatical/semantic role in *this* sequence, (c) a weighted blend of every other token's contribution (causally masked in autoregressive models).
- **Important:** these are the Transformer's direct output — an *intermediate representation*, **not the final answer**. Inside: L stacked layers, each containing self-attention and a feed-forward network.
- **Turning embeddings into task output.** A small *task head* sits on top:
 - Generation: project $e'^{(n)}$ onto the vocabulary $\rightarrow$ softmax $\rightarrow$ sample a token (Act 4).
 - Classification: project $e'^{([\text{CLS}])}$ onto class labels $\rightarrow$ softmax $\rightarrow$ pick a class.

Self-Attention, Step by Step

Five steps, one worked FOMC example

Self-Attention: The Key Idea

- For each position i in the sequence, ask: “*which other positions are most relevant to position i ?*”
- Compute a set of weights α_{ij} (with $\sum_j \alpha_{ij} = 1$) and form a new representation as a *weighted sum* of all positions.
- **Crucial properties:**
 - Every token can attend to every other token in one step (no recurrence)
 - Weights are computed dynamically from the input, not fixed
 - Operation is parallelizable across positions $\Rightarrow$ trains efficiently on GPUs
- This single operation replaces a tower of recurrent steps and is the central piece that made the Transformer work.

Step 1 — Project to Q, K, V

- For each input embedding $e^{(i)} \in \mathbb{R}^d$, compute three projections:

$$Q_i = W_Q e^{(i)} \quad (\text{query})$$

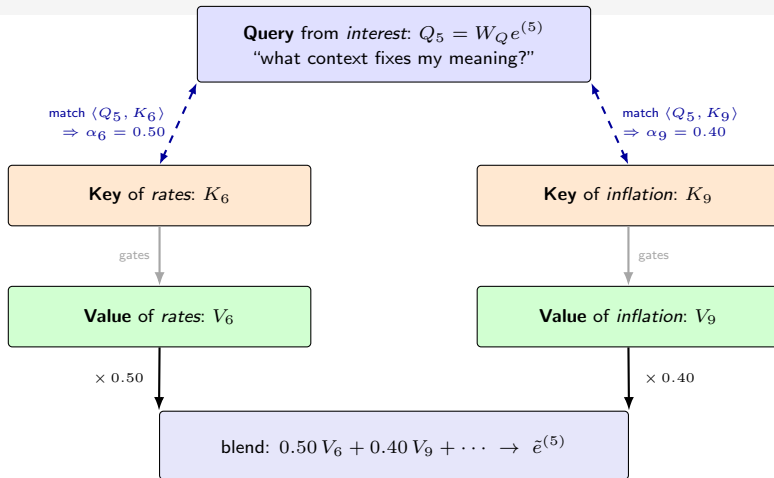
$$K_i = W_K e^{(i)} \quad (\text{key})$$

$$V_i = W_V e^{(i)} \quad (\text{value})$$

where $W_Q, W_K, W_V \in \mathbb{R}^{k \times d}$ are learned matrices and typically $k \leq d$.

- Mnemonic** (information retrieval):
 - Q_i is the *question* that token i asks: “what context do I need?”
 - K_j is the *description* that token j advertises: “here’s what I am about”
 - V_j is the *actual content* that j contributes if matched
- What W_Q, W_K, W_V are trained to achieve:** shape the projections so that the dot product $\langle Q_i, K_j \rangle$ is high when V_j ’s content is useful to what the task needs at position i , and low otherwise. The model discovers which pairings matter from the loss; it is never told in advance.
- Why three separate projections?** Separating roles enables *asymmetric* attention: *rates* can ask a question (Q) that matches *inflation*’s advertisement (K) without *inflation* having to ask the same question. The next slide shows the three roles in action.

Q/K/V at a Glance — Match Sets the Weight, the Weight Reads the Value



Three roles, kept separate. The *query*–*key* match (dashed) only decides *how much* to read; what gets read is the *value*. Asymmetry matters: *interest* queries *rates* without *rates* querying back. W_Q, W_K, W_V are all learned from the loss — nobody hand-codes “attend to rates.”

The Retrieval Analogy: A Soft Database Lookup

Attention is a search engine that returns a weighted blend instead of one hit.

Retrieval system	Self-attention	At position i
search query	query vector	$Q_i = W_Q e^{(i)}$: “what do I need?”
document tags / index	key vectors	$K_j = W_K e^{(j)}$: “what I am about”
document contents	value vectors	$V_j = W_V e^{(j)}$: what is actually read
relevance ranking	scaled dot products	$\langle Q_i, K_j \rangle / \sqrt{k}$
click the top result	softmax blend	read <i>all</i> results, weighted

- Every position plays all three roles at once: it queries the others, advertises itself, and delivers content when matched.
- The “soft” part — a convex combination rather than a hard top-1 pick — keeps the whole lookup differentiable, so it can be *learned* end-to-end by gradient descent.
- Preview: the same query–key logic powers document retrieval in RAG (Lec08) — there the “keys” are your corpus.

Step 2 — Scaled Dot-Product Scores

- For each pair (i, j) , measure the compatibility between query Q_i and key K_j by their dot product, scaled:

$$s_j^{(i)} = \frac{\langle Q_i, K_j \rangle}{\sqrt{k}}.$$

- Why divide by $\sqrt{k}$?** As the dimension k grows, dot products tend to grow too, pushing softmax into saturation (one term dominates, gradients vanish). Scaling by $\sqrt{k}$ keeps the variance of $s_j^{(i)}$ around 1.
- In autoregressive models, scores to future positions ($j > i$) are set to $-\infty$ before the softmax — the **causal mask** (more in Step 3).
- These raw scores are the “relevance ranking” of the retrieval analogy — one number per candidate position, before normalization.

Step 3 — Softmax → Attention Weights

- Convert scores into a probability distribution over positions:

$$\alpha_j^{(i)} = \frac{\exp(s_j^{(i)})}{\sum_{r=1}^n \exp(s_r^{(i)})}.$$

- Properties:
 - $\alpha_j^{(i)} \geq 0$ and $\sum_j \alpha_j^{(i)} = 1$
 - Concentrated weights $\Rightarrow$ “hard” attention to a few positions
 - Diffuse weights $\Rightarrow$ “soft” attention spread across many
- **Causal masking** (autoregressive models): when predicting position $i + 1$, set $s_j^{(i)} = -\infty$ for $j > i$ before the softmax. This prevents the model from peeking at the future during training.
- **Encoder models** (BERT) use bidirectional attention: every token sees every other.
- **Why softmax?** We want the output to be a *convex combination* of value vectors — weights that sum to 1, all non-negative. Softmax converts any real-valued score vector into exactly such a probability distribution, enabling an interpretable weighted average of content across the sequence.

Step 4 — Weighted Sum of Values

- Given the attention weights $\alpha_j^{(i)}$, the attention output for position i is the weighted sum of all value vectors:

$$\text{AttnVec}_i = \sum_{j=1}^n \alpha_j^{(i)} V_j \in \mathbb{R}^k.$$

- Interpretation:** token i “reads” content from all positions in the sequence, but pays most attention to those whose keys best match its query.
- Long-range, short-range, and same-position interactions all happen in the same step — contrast with RNNs where information has to propagate through many time steps.
- Note: V_j is what is being passed around. K_j only decides *how much* to pass; the content delivered is V_j .
- Why a weighted sum rather than a hard lookup?** Instead of selecting a single “most similar” token, the model retrieves a *blend*: for *interest* in a monetary-policy sentence, it mixes V_{rates} , $V_{\text{inflation}}$, and V_{raised} into one vector that reflects all three contexts simultaneously — richer than any single lookup could provide.

Step 5 — Project Back to Dimension d

- The attention output lives in $\mathbb{R}^k$, but we need a vector in $\mathbb{R}^d$ to feed the next layer. Project back with a learned matrix $W_O \in \mathbb{R}^{d \times k}$:

$$\tilde{e}^{(i)} = W_O \text{AttnVec}_i.$$

- This gives the contextualized representation of token i *after* self-attention.
- **Why project back to $\mathbb{R}^d$?** In multi-head attention each head operates in a smaller subspace $\mathbb{R}^k$ (with $k = d/H$) to keep compute manageable. Projecting back with W_O after concatenating heads restores the common dimension d , so the result flows into the next layer without any shape change.
- **Five-step recap:**
 1. Project: Q_i, K_i, V_i
 2. Score: $s_j^{(i)} = \langle Q_i, K_j \rangle / \sqrt{k}$
 3. Softmax: $\alpha_j^{(i)}$
 4. Weighted sum: $\text{AttnVec}_i = \sum_j \alpha_j^{(i)} V_j$
 5. Output projection: $\tilde{e}^{(i)} = W_O \text{AttnVec}_i$
- All matrices (W_Q, W_K, W_V, W_O) are learned by backpropagation from a task loss.

Worked Example — Part 1

Sentence: “The central bank raised interest rates amid rising inflation.”

Tokenization (9 word tokens, numbered left to right):

The central bank raised interest rates amid rising inflation
1 2 3 4 5 6 7 8 9

Goal: produce $e'^{(5)}$, the contextualized vector for *interest* — one that encodes how the word is used *in this sentence* (the interest of monetary-policy rates, not “curiosity” or “financial stake”).

- **Attention score** $s_j^{(5)} = \langle Q_5, K_j \rangle / \sqrt{k}$ measures how much *interest* should listen to token j . Suppose, after scaling, the model finds:

Token j	$s_j^{(5)}$
rates ($j = 6$)	2.3
inflation ($j = 9$)	1.9
raised ($j = 4$)	1.1
bank ($j = 3$)	0.5
the ($j = 1$)	-0.2

Intuitively: *interest* is most strongly tied to *rates* and *inflation* — the words that fix its meaning here.

Worked Example — Part 2

- **Softmax** $\rightarrow$ **attention weights**. $\alpha_j^{(5)}$ is the *fraction* of attention *interest* pays to token j — a probability, with $\sum_j \alpha_j^{(5)} = 1$:

$$\alpha_6^{(5)} \approx 0.50, \quad \alpha_9^{(5)} \approx 0.40, \quad \alpha_4^{(5)} \approx 0.05, \quad \dots$$

50 % of the attention goes to *rates*, 40 % to *inflation*, a sliver to everything else.

- **Weighted sum of value vectors** (what the token actually “reads”):

$$\text{AttnVec}_5 \approx 0.50 V_6 + 0.40 V_9 + 0.05 V_4 + \dots$$

- **Project back**: $\tilde{e}^{(5)} = W_O \text{AttnVec}_5$.
- **Result**: $\tilde{e}^{(5)}$ is a vector in $\mathbb{R}^d$ that represents “*interest, in a context dominated by rates and inflation*” — exactly the meaning we want for monetary-policy text.
- Swap the sentence to “The discount rates applied to pension funds were revised” and the same machinery shifts its reading: *discount* and *pension* become the high-weight sources, and the vector lands in discounting/pensions territory instead.
- The same computation runs in parallel for every position i , producing $\tilde{e}^{(1)}, \dots, \tilde{e}^{(9)}$ — every

How Q , K , V Are Learned

- **What is trainable (the “controls” θ):** in every layer, W_Q, W_K, W_V, W_O , plus the FFN weights W_1, W_2, b_1, b_2 , LayerNorm parameters, the token embedding matrix $\mathcal{E}$, and the output head W_{vocab} . All are initialized randomly and updated by gradient descent on a **single global loss**.
- **Notation:** $t^{(i)}$ is the i -th token of a training sequence s . The sum $\sum_s \sum_i$ runs over every position i in every sequence s in the corpus.

- **Common training objectives:**

- **Autoregressive LM** (GPT-style):

$$\mathcal{L}(\theta) = - \sum_s \sum_i \log P_{\theta}(t_s^{(i+1)} \mid t_s^{(1:i)}).$$

- **Masked LM** (BERT-style): hide tokens at random and predict them from bidirectional context.
 - **Classification fine-tuning:** cross-entropy on a sentence-level label.
- **One loss, all parameters.** Backpropagation sends gradients through softmax, dot products, Q/K/V projections, residuals, LayerNorm, *and the FFN* — one backward pass updates every parameter together. There is no separate training signal for attention vs. the feed-forward block.
- Nothing tells the model “attend to neighbors” or “ignore stopwords” — it discovers attention patterns from data.

Multi-Head Attention

- One (Q, K, V) trio captures only one “view” of the sequence. **Multi-head attention** runs H such trios in parallel and concatenates their outputs.
- Each head h has its own projection matrices $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$, all of dimension $k = d/H$.
- After running H attention computations independently, concatenate and project:

$$\text{MultiHead}_i = W_O^* \left[\text{AttnVec}_i^{(1)}; \dots; \text{AttnVec}_i^{(H)} \right].$$

- **Do heads have different objectives?** *No — all H heads share the same training loss* (e.g. next-token cross-entropy). What differs is their random initialization. Specialization is **emergent**: under one global loss, H parallel feature detectors diversify because identical copies of the same head would be redundant.
- **Observed specializations in financial text:**
 - Head 1: monetary-policy relations (*inflation* $\leftrightarrow$ *interest*)
 - Head 2: temporal phrases (*Q1, next month, by year-end*)
 - Head 3: hawkish vs. dovish stance markers

Nobody told the heads what to look for — they were not assigned separate objectives.

- Total parameter cost is essentially the same as a single-head attention with the full dimension d

Thinkbox: Attention Entropy as a Measurement Diagnostic

The weight distribution itself carries information about measurement quality.

- Position i 's weights $\alpha^{(i)}$ form a probability distribution; its Shannon entropy $H(\alpha^{(i)}) = -\sum_j \alpha_j^{(i)} \log \alpha_j^{(i)}$ measures how *focused* the reading is.
- $H \approx 0$: meaning pinned down by a few strong context words — 2022-style tightening statements, where *inflation* attends to *raised*, *persistent*, *mandate*.
- $H \approx \log n$: attention spread thin — the contextual vector approaches a global average. 2019-style “crosscurrents” statements: *inflation* attends diffusely to *employment*, *growth*, *uncertainty*, ...

High-entropy passages produce noisier stance scores — measurement-error variance that *moves with the state of the economy*. If you regress on Transformer-based stance measures, attention entropy is a candidate flag for heteroskedastic measurement error. A PBOC wrinkle to discuss: boilerplate political formulas share every passage with the substantive policy words — does that push the attention entropy of terms like “inflation” systematically higher in PBOC reports than in FOMC statements?

Bridge: One Head Is a Building Block

We can now compute a context-aware vector — once.

- One attention pass mixes information *across* positions. On its own it is still linear in the values, blind to word order, and single-pass.
- **T3b wraps the head into a layer:** positional encodings (order), residual connections + LayerNorm (trainability at depth), a feed-forward network (nonlinearity — and, it turns out, the model's stored knowledge) — then stacks the layer L times and chooses among three architectural variants, ending with a full research case on 10-K filings.

*Arc: T1 FRAME → T2 REPRESENT → **T3 ATTEND** → T4 PRETRAIN → T5 MEASURE.*

AI for Economic Research: Dynamic Models, Language, and Agents

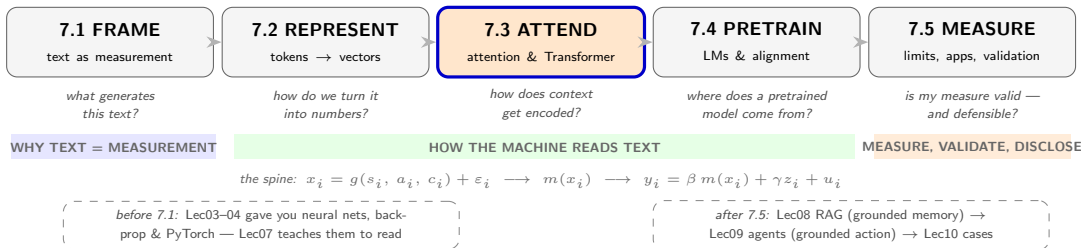
Lecture 7.3b: The Transformer

Zhigang Feng

2026-07-07 05:00:15

Lecture 7 in One Map

you are here ▼



Route: wrap the attention head into a full layer (part 2/2 of ATTEND) — position, residual, FFN → stack L layers → pick among three variants → read 77,000 10-Ks with the same machinery.

Completing the Layer

Position, residual safety nets, and the network that stores the knowledge

Positional Encodings: Why They Are Needed

- **Problem:** self-attention is *permutation-equivariant* in its inputs. The tokens $\{\textit{The, Fed, raised, rates}\}$ give the same internal state whether we read “*The Fed raised rates*” or the scrambled “*Rates raised the Fed*” — yet the economic meaning is opposite (agent and patient are reversed). We must inject position.
- **Solution:** add a position vector $p(i)$ to each token embedding before the first layer:

$$e^{(i)} = \mathcal{E}(t^{(i)}) + p(i).$$

- Three families of position encodings dominate practice. The choice matters most for *long contexts* and *length extrapolation*.
- **Take-away:** a Transformer must be *told* where each token is. The mechanism is simple but indispensable.

Sinusoidal, Learned, and Rotary (RoPE)

- **Sinusoidal** (Vaswani et al., 2017) — fixed, no parameters:

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{2i/d}), \quad \text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{2i/d}).$$

Multiple frequencies $\Rightarrow$ relative offsets become linear functions; theoretically extrapolates beyond training length.

- **Learned** (BERT, original GPT) — positions $1, \dots, n_{\max}$ each get a learnable vector. Simple; *cannot* extrapolate past $n_{\max}$.
- **Rotary (RoPE)** — modern default in LLaMA, GPT-NeoX, Qwen, Mistral, DeepSeek. Apply a position-dependent *rotation matrix* to Q and K :

$$\langle R_{\theta_i} Q_i, R_{\theta_j} K_j \rangle = f(Q_i, K_j, i - j).$$

Position information is baked into attention scores and depends only on *relative* offset $i-j$ — excellent length extrapolation.

- **ALiBi** (a close cousin) adds a linear distance penalty directly to attention scores; very simple, strong long-context behavior.

Position Encodings: Trade-Off Table

Scheme	Params	Where applied	Extrapolates?	Modern use
Sinusoidal	0	input embedding	yes (in theory)	rare today
Learned absolute	$n_{\max} \cdot d$	input embedding	no	BERT, early GPT
RoPE	0	inside Q, K	yes (with scaling)	LLaMA, Qwen, Mistral
ALiBi	H	attention scores	yes	some long-context models

- **Take-away:** a Transformer must be *told* where each token is. Modern LLMs prefer *relative* schemes (RoPE / ALiBi) because they handle 32k+ context windows gracefully.

For long-document economics — complete 10-Ks (50k+ tokens), IMF country reports, decade-scale central-bank corpora — prefer RoPE/ALiBi-based models. A fixed learned-absolute model simply cannot read past its training length.

Residual Connection and Layer Normalization

- After the attention output $\tilde{e}^{(i)}$, the layer combines it with the original input via a *residual connection* and then a *layer normalization*:

$$u^{(i)} = \text{Norm}(e^{(i)} + \tilde{e}^{(i)}).$$

- **Why residual?**

- Preserves the original information — attention can only *add* to it, never destroy it
- Provides a clean gradient path back through dozens of layers (the trick that made deep networks trainable)

- **Why layer normalization?**

- Normalizes activations across the feature dimension to mean 0, variance 1
- Adds learnable scale γ and shift β
- Stabilizes optimization dynamics, especially for very deep stacks

Feed-Forward Network (MLP)

- **Division of labor inside a layer.** Attention mixes information *across* tokens; the FFN then transforms *each* token individually given what it just gathered. Attention = the meeting room; FFN = desk work after the meeting.
- **Why it exists.** Attention alone is a linear map of value vectors. The FFN supplies the *non-linear* transformation that lets the network represent arbitrary functions of a token's contextualized embedding.
- After attention + residual + norm, each position is passed independently through a small feed-forward network:

$$f^{(i)} = W_2 \sigma(W_1 u^{(i)} + b_1) + b_2.$$

- **Shape conventions:** $W_1: d \rightarrow 4d$ (expand), activation σ (ReLU / GELU / SwiGLU), $W_2: 4d \rightarrow d$ (contract). The $4d$ expansion is where most of an LLM's parameters live.
- **A useful reading — key-value memory** (Geva et al. 2021): rows of W_1 act as *pattern detectors* ("keys": a collocation, a topic, an entity type); the ReLU selects which patterns fired; the matching columns of W_2 write back the associated content ("values"). Roughly 2/3 of a layer's parameters sit here ($8d^2$ vs. attention's $4d^2$): attention *moves* information to the right

One Layer — Putting It All Together

- A single Transformer layer can be summarized by the flow:

$$e^{(i)} \xrightarrow{\text{Self-Attention}} \tilde{e}^{(i)} \xrightarrow{\text{Res+Norm}} u^{(i)} \xrightarrow{\text{MLP}} f^{(i)} \xrightarrow{\text{Res+Norm}} e'^{(i)}.$$

- Operationally:

$$\tilde{e}^{(i)} = W_O \sum_j \alpha_j^{(i)} V_j,$$

$$u^{(i)} = \text{Norm}(e^{(i)} + \tilde{e}^{(i)}),$$

$$f^{(i)} = W_2 \sigma(W_1 u^{(i)} + b_1) + b_2,$$

$$e'^{(i)} = \text{Norm}(u^{(i)} + f^{(i)}).$$

- Same $(\mathbb{R}^d)^n$ shape in and out: a layer is a structured non-linear update — which is exactly what lets us stack it.

After the Transformer: Where We Are

- **What went in:** a sequence of n tokens, converted to embeddings + positional codes, $[e^{(1)}, \dots, e^{(n)}]$ with each $e^{(i)} \in \mathbb{R}^d$.
- **What comes out of the top layer:** n *contextualized* vectors $[e'^{(1)}, \dots, e'^{(n)}]$, same shape, same dimension. Each $e'^{(i)}$ encodes, in its d coordinates,
 - (a) the identity of token $t^{(i)}$,
 - (b) its syntactic / semantic role in *this* sequence,
 - (c) what every other token contributed to its meaning (causally masked in GPT-style models).
- **What we have NOT yet done:** no token generated, no class assigned, no number returned. Contextualized embeddings are an **intermediate data structure**, not the final answer.
- **What comes next:** a small *task head* on top — a vocabulary projection (generation) or a class projection (classification) — turns the embeddings into the output we want.

Stacking L Layers

- The output of one layer becomes the input of the next:

$$[e^{(1)}, \dots, e^{(n)}]_{\text{layer } \ell+1} = [e'^{(1)}, \dots, e'^{(n)}]_{\text{layer } \ell}.$$

- After L layers, each token's vector has accumulated context from the entire sequence (subject to causal masking, in the autoregressive case).
- **Different layers learn different things** (empirically):
 - Lower layers: surface syntax, local agreement, morphology
 - Middle layers: phrase- and clause-level structure
 - Upper layers: discourse, factual associations, task-relevant abstractions
- **What we do with the top layer (“task heads”).**
 - **Generation:** logits = $W_{\text{vocab}} e'^{(n)}$, then $P(\text{next} = w) = \text{softmax}(\text{logits})_w$. Training loss:
 $\mathcal{L} = -\sum_i \log P(t^{(i+1)} \mid t^{(1:i)})$ — the same cross-entropy that shaped every weight in the stack during pretraining.
 - **Classification:** logits = $W_{\text{class}} e'^{([\text{CLS}])}$, then $P(\text{class} = c) = \text{softmax}(\text{logits})_c$. Training loss:
 $\mathcal{L} = -\sum \log P(y_{\text{true}} \mid x)$ on a labeled set (fine-tuned encoders: BERT, CentralBankRoBERTa, ...).

Why the Transformer Displaced the RNN

Two compounding advantages — each with an economic hook.

1. Fully parallel training. An LSTM must compute step t after step $t-1$: n tokens means n serial steps. Attention over the whole sequence is two matrix multiplications, $\text{softmax}(QK^\top/\sqrt{k})V$ — computed for all positions at once on a GPU. This is the engineering fact behind “pretraining on billions of tokens became possible after 2017”: BERT’s 3.3×10^9 -token pretraining was simply not feasible in serial mode.

2. $O(1)$ paths for long-range dependencies. In a recurrent net, position j ’s influence on a distant position i must survive $i-j$ intermediate states — the signal decays. Attention connects any two positions *directly*. Concretely: Fed and PBOC policy reports state economic conditions in part 1 and the policy orientation pages later — reading the guidance *against* the conditions requires exactly these long jumps.

Post-script: state-space models (e.g. Mamba, Gu & Dao 2023) revive recurrence with linear cost in sequence length — worth watching for full-filing work, at some cost in exact global attention.

From Output Embeddings to the Next Token

- For autoregressive generation we focus on the *last* output vector $e'^{(n)}$.
- Project it onto the vocabulary with a linear layer:

$$\text{logits} = W_{\text{vocab}} e'^{(n)} + b_{\text{vocab}}, \quad W_{\text{vocab}} \in \mathbb{R}^{M \times d}.$$

- Convert logits to probabilities with softmax:

$$P(t^{(n+1)} = w \mid t^{(1)}, \dots, t^{(n)}) = \frac{\exp(\text{logits}_w)}{\sum_j \exp(\text{logits}_j)}.$$

- **Sampling strategies:** greedy, top- k , top- p (nucleus), temperature scaling. (Unpacked in T4.)
- Append the chosen token to the context and repeat. **That** is how an LLM “generates text”.

Real LLM Scale

Quantity	Symbol	GPT-2	GPT-3	Modern (2024–26)
Layers	L	12–48	96	60–120
Hidden dim	d	768–1600	12288	4096–16384
Context length	n	1024	2048	32k–1M
Vocabulary	M	50k	50k	50k–256k
Parameters	—	0.1–1.5B	175B	70B–>1T

- **Modern open-weight examples (2024–2026):** LLaMA-3 (8B, 70B, 405B), DeepSeek-V3 (671B MoE), Qwen-2.5, Mistral.
- **Modern closed examples:** GPT-4/5, Claude 3/4, Gemini (sizes undisclosed).
- All share the same basic architecture; differences are scale, training data, and tuning — the subject of Act 4.

Transformer vs. Word2Vec

Property	Word2Vec	Transformer
Representation	1 static vector / word	contextual vector / token
Architecture	2-layer Skip-gram / CBOW	multi-head attention + MLP, deep stack
Context scope	fixed local window (± 5)	full sequence, every token sees every token
Position info	none	explicit positional encodings
After training	lookup table; model discarded	model <i>is</i> the network used downstream
Typical size	$\sim 10^7$ params (CPU OK)	10^8 – 10^{12} params (GPU required)
Best for	similarity, feature inputs	generation, classification, QA, RAG

The big shift: from a frozen lookup table of meanings to a deep, dynamic model where meaning emerges from interaction with context.

Three Variants

Same block, three masking patterns — and three research uses

Three Variants of the Same Block

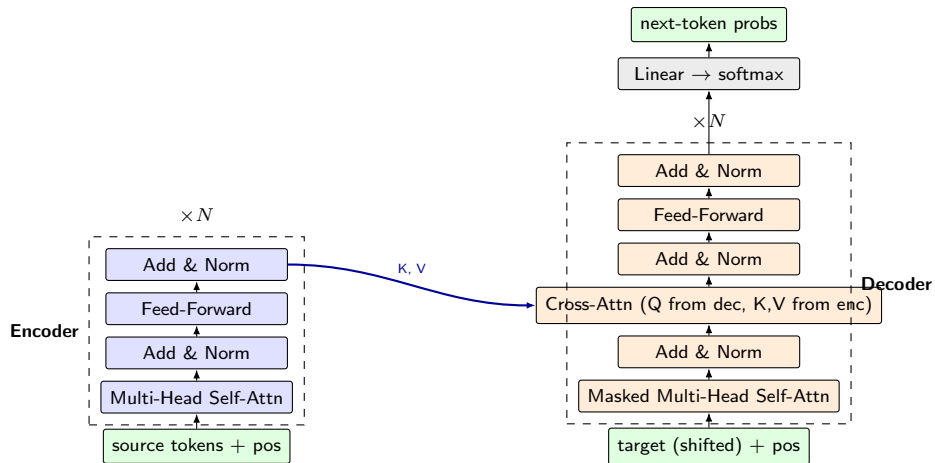
- The Transformer block is generic. The *masking pattern* and the *stack arrangement* determine what kind of model you get.
- **Encoder-only (BERT, RoBERTa, FinBERT, CentralBankRoBERTa)**
 - Bidirectional attention — every token sees every other.
 - Pretrained with *masked language modeling*: hide tokens, predict from both sides.
 - Best for: classification, embedding extraction, similarity. *Not* a generator.
- **Decoder-only (GPT, LLaMA, Claude, Qwen)**
 - Causal mask — each position only attends to itself and the past.
 - Pretrained with *next-token prediction*; autoregressive at inference (Act 4).
 - Best for: open-ended generation, in-context learning, chat. The dominant paradigm today.
- **Encoder–Decoder (T5, BART, original Vaswani 2017)**
 - Encoder reads source bidirectionally; decoder generates target with causal + cross-attention to encoder.
 - Best for: translation, summarization, **structured extraction** — e.g. turning free-form FOMC or PBOC minutes into (date, action, magnitude, guidance) database rows at a fraction of hand-coding cost.

Picking a Variant for Your Macro Project

Task	Pick	Why	Example model
Sentiment / hawk–dove score on FOMC text	encoder	need a fixed embedding	CentralBankRoBERTa
Topic / event classification	encoder	supervised fine-tune	FinBERT, RoBERTa
Document embeddings for similarity / RAG	encoder	pooled $e'([CLS])$	SBERT, BGE, E5
Summarize a 10-K / press release	enc–dec	input $\neq$ output	T5, BART
Structured extraction (minutes $\rightarrow$ rows)	enc–dec	learned text-to-schema map	T5, BART
Generate a forecast narrative	decoder	open-ended text	GPT-4, Claude, LLa
Q&A over central-bank corpora (RAG)	decoder + retriever	generative w/ context	Lecture 8

- **Rule of thumb.** If the output is a label or a vector, use an encoder. If the output is text, use a decoder. If you need to map one sequence to another and they have different lengths and structure, use encoder–decoder.

“Attention Is All You Need” (Vaswani et al., 2017) — the Architecture

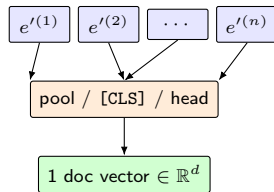


Same block, two arrangements. Decoder-only models drop the encoder and the cross-attention.

Token Embeddings vs. Document Embeddings (Bridge to Lec08)

- **What the Transformer gives us directly.** For an n -token passage, the top layer outputs n contextualized vectors $e'(1), \dots, e'(n) \in \mathbb{R}^d$ — *one per token*.
- **To compare whole passages we need one vector per passage.** Standard pooling moves:
 - **Mean-pool:** $\bar{e} = \frac{1}{n} \sum_i e'^{(i)}$ — simple, surprisingly robust.
 - **[CLS]-pool** (BERT family): use $e'^{([\text{CLS}])}$, trained to summarize the passage.
 - **Learned pooling head:** a small attention head over the token sequence, trained for the downstream objective.
- **Retrieval embedders** (Sentence-BERT, BGE, E5, voyage-3) take a pretrained transformer and fine-tune it with a *contrastive loss* on (anchor, positive, negative) triples: pull related passages together, push unrelated ones apart. The geometry is *learned for similarity*, not inherited from the

n token vectors



Lec08: ranks $\cos(\text{query}, \text{doc})$

Case: Reading 10-Ks at Paragraph Level

Kim, Muhn, Nikolaev & Zhang (2024) — every concept of this act, reused

From Tokens to Paragraphs: A Research Question

- **Question.** Which language inside a 10-K filing actually moves stock prices? Press releases, MD&A, risk factors, governance disclosures — which content do investors *react* to?
- **Paper.** Kim, Muhn, Nikolaev, Zhang (Booth WP, Nov. 2024), *Learning Fundamentals from Text*.
- **The move.** Same Q/K/V machinery from T3a — but the unit of analysis is a **paragraph**, not a word token. A 10-K is a sequence of ~ 400 paragraphs; treat each one as a “token” and let attention learn which paragraphs matter for the market.
- **Why this beats the standard playbook:**
 - Prior text-in-finance work collapses a 10-K to one number: sentiment (Loughran–McDonald 2011), uncertainty (Hassan et al. 2019). Information loss is severe.
 - Attention preserves *paragraph-level* granularity *and* learns relational importance directly from market reactions.
 - Trained attention weights are inspectable — you can read off which content the market focuses on.

Architecture: Two Attention Layers + MLP Head

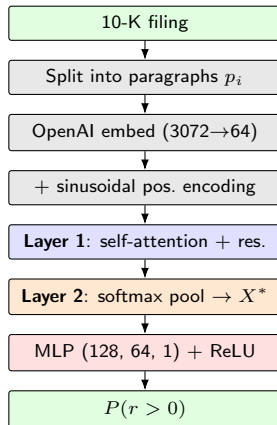
Pipeline (full universe of EDGAR 10-Ks, 1996–2023, ~77K filings, ~20.7M paragraphs):

1. Split filing $\rightarrow$ paragraphs $p_1, \dots, p_n$ (cap $n=1000$).
2. $e_i = \text{OpenAI text-embedding-3-large}(p_i)$, then truncate to first 64 dims (Matryoshka, Kusupati et al. 2022).
3. Stack $X = [e_1; \dots; e_n] \in \mathbb{R}^{n \times 64}$, add sinusoidal positional encoding.
4. **Layer 1** — Vaswani self-attention with residual: $\text{Attention}(X) \in \mathbb{R}^{n \times 64}$.
5. **Layer 2** — learned softmax pool to a doc vector:

$$X^* = \text{softmax}\left(\frac{W^\top \text{Attention}(X)^\top}{\sqrt{64}}\right) \text{Attention}(X)$$

with $W \in \mathbb{R}^{64}$ learnable; $X^* \in \mathbb{R}^{64}$.

6. MLP ($64 \rightarrow 128 \rightarrow 64 \rightarrow 1$), ReLU, sigmoid
 $\rightarrow P(r_{[-1, +30]} > 0)$.



Target: $\mathbb{I}\{r_{[\text{file}-1, \text{file}+30]} > 0\}$. Rolling 6/2/1-yr train/val/test.

Every Concept from This Act, Reused

Slide concept (this act)	Paper usage
Q, K, V projections (T3a)	Same, but over <i>paragraphs</i> not tokens
Scaled softmax $\text{softmax}(QK^\top / \sqrt{d})V$ (T3a)	Used verbatim in Layer 1
Sinusoidal positional encoding (T3b)	Same Vaswani formula, scaled by embedding norm
Residual connection (T3b)	Added to Layer 1 output (paper fn. 13)
Cosine similarity (T2b)	Importance score: $\cos(e_i, X^*)$
Encoder-style training	Supervised on returns , not next-token

- **Methodological pivot.** A standard LLM trains attention with an unsupervised objective (next-token cross-entropy). This paper trains the *same* attention machinery against a **financial target** (binary positive return). The attention weights then read directly as economic importance.
- **Importance score for paragraph i in document t :**

$$\text{Importance}_{ikt} = \frac{X_{it}^* \cdot e_{ikt}}{\|X_{it}^*\| \|e_{ikt}\|}$$

A paragraph is important when its semantic content aligns with the document's attention-weighted representation.

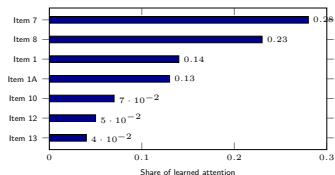
Empirical Findings

Predictive performance (avg AUC, 20 yearly test sets, baseline 0.50):

Model	AUC	Δ vs base
Attention (paper)	0.5599	+6.0 pp
MLP (no attn)	0.5328	+3.3 pp
Logit	0.5197	+2.0 pp

Sharpe (1-month-ahead returns): **1.56** attention, 1.28 MLP, 1.08 logit.

Attention share by 10-K Item (stylized; learned weights):



Top: Item 7 MD&A, Item 8 Financials. Bottom: Item 13 Relations, Item 12 Ownership, Item 10 Governance.

Topics inside MD&A (Item 7):

- *Most attended*: segment information, financial performance, liquidity, recent M&A.
- *Least attended*: forward-looking statements, accounting pronouncements, **corporate governance**, **sustainability / CSR**.

Surprise. ESG and corporate governance — dominant in public discourse — rank *lowest* by market reaction.

Two causal results.

- SEC *Modernization of Reg S-K* (treatment) raised Item 7 vs Item 8 (control) importance — regulation can *enhance* relevance.
- Firms strategically place bad-news / low-profitability content **later** in MD&A (Bloomfield 2008; Cohen et al. 2020) — the DGP's a_i , visible in document structure.

Bridge: The Architecture Is Fixed — Capability Is Not

Same block everywhere; the leverage now moves to training.

- Kim et al. shows the pattern travels: change the “token” (paragraphs of a filing, sentences of a speech, stocks in a portfolio — cf. Gabaix–Koijen–Richmond–Yogo’s asset embeddings) and supervise attention on an economic target — the weights become an inspectable measure.
- But nothing so far explains where a model that already *reads* comes from. GPT-2 to GPT-4 is essentially the same architecture — what changed is pretraining scale, data, and alignment.
- **Act 4:** self-supervised pretraining, prompting / fine-tuning / LoRA, and the SFT + RLHF pipeline that turns a text-completer into an assistant — plus what alignment does to measurement.

Arc: T1 FRAME → T2 REPRESENT → T3 ATTEND → T4 PRETRAIN → T5 MEASURE.

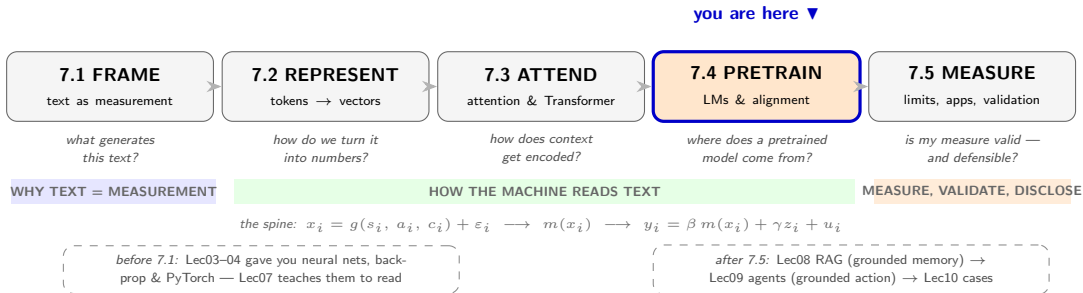
AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 7.4: Pretrained Language Models and Alignment

Zhigang Feng

2026-07-07 05:00:23

Lecture 7 in One Map

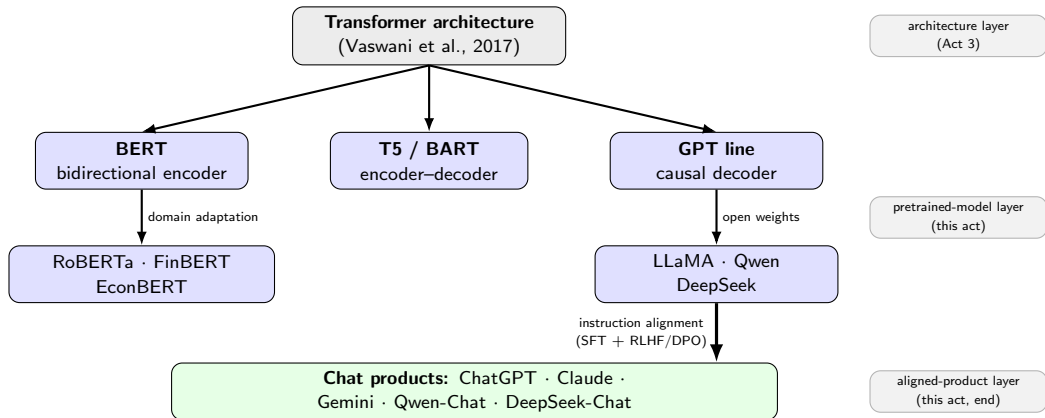


From Architecture to Model

Products vs. architectures: a three-layer family tree

Products vs. Architectures: The Family Tree

ChatGPT is not a model family — it is the aligned product of one. Three layers to keep straight:



Encoders (left branch) are the measurement workhorses — classification, embeddings, similarity. **Decoders** (right

branch) generate; alignment turns them into assistants. Calling ChatGPT means using all three layers at once: decoder

Pretraining: Self-Supervised Language Modeling

- **Data:** hundreds of billions to trillions of tokens of natural text and code (web pages, books, papers, code repositories, filings, news, ...).
- **Objectives:**
 - **Causal LM** (GPT family): $\mathcal{L}(\theta) = - \sum_s \sum_i \log P_{\theta}(t_s^{(i+1)} | t_s^{(1:i)})$. *In words:* at every position in every training sequence, the model outputs a vocabulary distribution; we penalize it by the negative log-probability of the token that *actually* appeared.
 - **Masked LM** (BERT family): hide a random subset of tokens and predict them bidirectionally, $\mathcal{L}(\theta) = - \sum_{i \in \text{masked}} \log P_{\theta}(t^{(i)} | t^{(\setminus i)})$.
- **This is maximum-likelihood estimation** of a conditional distribution. The novelty vs. classical statistics is the parameterization (10^{11} -parameter neural network) and the scale of the data — not the statistical objective.
- **Optimization:** Adam/AdamW, large batches, warmup + cosine LR, gradient clipping, mixed precision. **Compute:** weeks–months on thousands of accelerators, 10^{23} – 10^{25} FLOPs. No human labels — the next token *is* the label.

Two Pretraining Objectives: MLM vs. CLM

Fill in hidden words from both sides, or predict strictly left to right — the objective determines what the model is good for.

	BERT (masked LM)	GPT (causal LM)
Objective	mask ~15% of tokens; predict them from <i>both</i> sides	predict every next token from the left context
Context direction	bidirectional	causal (left-to-right)
Training signal	masked positions only	every position
Natural strength	understanding: disambiguation, classification, embeddings	generation: continuation, summarization, dialogue
Typical econ use	stance scoring, 10-K classification, similarity, retrieval	summaries, structured extraction, zero-shot labeling
Adaptation cost	low: linear head on [CLS]	prompt design or instruction tuning

Where bidirectionality pays: disambiguation needs the *right-hand* context too — *rate* (policy instrument vs. unemployment statistic), *liability* (accounting line vs. everyday negative). For pure measurement tasks on fixed text, the older BERT family is often still the better *instrument*: cheaper, 5/28

Generating the Next Token

- **After pretraining** we have a trained θ . Generation uses that θ to extend a prompt.
- Given a prompt $t^{(1)}, \dots, t^{(n)}$, run the Transformer to obtain $e'^{(n)}$.
- Compute logits and softmax:

$$P(t^{(n+1)} = w \mid t^{(1)}, \dots, t^{(n)}) = \text{softmax}(W_{\text{vocab}} e'^{(n)})_w.$$

- **Pick** a token according to a sampling strategy, **append** it to the context, and **repeat** until a stop token, a length limit, or an end-of-sequence marker.
- **Cost:** naively, generating n tokens is $O(n^2)$ in attention; practical implementations cache keys and values.

Tokens, Cost, and Reproducibility

Frontier-LLM economics (list prices checked 2026-07):

Model	In \$/M	Out \$/M
Claude Opus 4.8	5	25
GPT-5.4	2.50	15
Gemini 3.1 Pro	2	12
Qwen / DeepSeek / LLaMA (local)	GPU-hours, not API fees	

Frontier context windows now span $\sim 200\text{k}$ – 1M tokens. This is a *dated snapshot*: list prices, context limits, and the flagship generation move every quarter. **Lock the snapshot — model id, prices, and date — the day you run the study.**

What this means for measurement:

- Yin–Vu–Persico (T5a/T5b) run 18,797 O*NET tasks $\times$ 3 frontier annotators $\approx$ 56,000 calls. At $\sim 2\text{k}$ tokens/call $\Rightarrow \sim 110\text{M}$ tokens per provider. *Per-call* cost is trivial; *per-study* cost is not.
- **Reproducibility budget.** A pinned model version + frozen prompt + fixed seed (where supported) + recorded date is one “equipment item.” Treat it as you would a survey instrument.
- **Three things to archive** so a referee can re-run you:
 - the exact API call (model id, temperature, seed),
 - the inputs (hashes),
 - the raw outputs *before* any post-processing

Sampling Strategies

- **Greedy:** pick $\arg \max_w P(w \mid \cdot)$. Deterministic, but often repetitive and bland.
- **Top- k sampling:** restrict to the k most likely tokens, renormalize, then sample. Cuts off the long tail of low-probability nonsense.
- **Top- p (nucleus) sampling:** restrict to the smallest set of tokens whose cumulative probability exceeds p (e.g. 0.9). Adapts to how peaked the distribution is.
- **Temperature τ :** rescale logits by $1/\tau$ before softmax.
 - $\tau \rightarrow 0$: deterministic, like greedy
 - $\tau = 1$: model's natural distribution
 - $\tau > 1$: more diverse, more creative, more risky
- **For empirical research, set $\tau = 0$ (and report it).** Reproducibility is more valuable than creativity when measuring something. For generative tasks (drafting, synthetic-text experiments), $\tau \in [0.7, 1.0]$ with top- $p \in [0.9, 0.95]$ is the usual compromise — but record the parameters and seed even then.

Steering Without Training

Prompting, in-context learning, and schema-constrained output

Prompting: Zero-Shot, Few-Shot, In-Context Learning

Zero-shot. No examples in the prompt.

```
Classify this FOMC sentence as  
hawkish, dovish, or neutral:  
"Inflation remains elevated and  
the Committee is highly atten-  
tive."  
Label:
```

→ model: hawkish. Works when the task name is in the pretraining distribution; brittle on rare or domain-specific tasks.

Few-shot. 2–8 exemplars in the prompt.

```
"Inflation is too high."  
Label: hawkish  
"Risks are tilted to the down-  
side."  
Label: dovish  
"...highly attentive."  
Label:
```

→ accuracy jumps without retraining — the model infers the label space and the decision rule from the demonstrations.

In-context learning (ICL).

- No gradient updates.
- Demonstrations act as a temporary “task vector” inside the forward pass.
- Empirically: accuracy scales with the number and quality of exemplars; ordering, label format, and class balance all matter.
- This is the workhorse pattern for LLM-as-rater measurement (T5a/T5b, Eloundou et al.).

In-Context Learning as Implicit Bayesian Inference

- **Xie, Raghunathan, Liang, Ma** (ICLR 2022; arXiv 2111.02080) frame ICL as Bayesian inference over a latent “task” variable τ :

$$P(y_{\text{target}} \mid x_{\text{target}}, \text{demos}) = \int P(y_{\text{target}} \mid x_{\text{target}}, \tau) \underbrace{P(\tau \mid \text{demos})}_{\text{posterior over task}} d\tau.$$

The prompt’s demonstrations *update* the model’s beliefs about *which* task to perform.

- **Three predictions, all observed empirically:**
 - More (relevant) demonstrations $\Rightarrow$ posterior $P(\tau \mid \text{demos})$ concentrates on the intended task $\Rightarrow$ accuracy rises.
 - Demonstration *ordering* and *label format* are not nuisances — they shape the posterior.
 - Out-of-distribution exemplars push the posterior toward the wrong τ ; ICL accuracy collapses.
- **Take-away.** *Your prompt is your prior.* The very mechanism that lets a frontier LLM adapt to a new economic measurement task without retraining is the same mechanism that produces sycophancy, prior-contamination, and template sensitivity. Lec09 returns to this as the *context dilemma* — the conditioning that helps also biases.

Structured Output: JSON as a Measurement Tool

Free-text answers are unstable. A rubric that asks for “hawkish / dovish / neutral” may produce Hawkish., ‘hawkish’, The label is hawkish., or worse. Post-processing such variation is brittle and undermines reproducibility.

Solution. Force the model to emit a fixed-schema JSON object. Every provider exposes a “structured output” / “tool use” API that constrains decoding (logit masking over disallowed tokens) so the output is *guaranteed* to parse.

Mechanism. The model is still next-token predicting. The provider’s API simply masks tokens that would violate the schema — the next-token distribution is restricted to syntactically valid continuations.

Why it bridges to Lec09. An “agent” is just an LLM whose schema names *tools to call*, not labels to output. The same machinery.

Sentiment-extraction schema:

```
{
  "stance": "hawkish|dovish|neutral",
  "confidence": 0.0,
  "evidence_phrase": ""
}
```

Tool-call schema (Lec09 preview):

```
{
  "tool": "fred_query",
  "args": {
    "series": "UNRATE",
    "start": "2020-01"
  }
}
```

For econ research: schemas *enforce* the rubric (no off-rubric labels), expose the model’s evidence (auditable), and let you compute Cohen’s κ on parseable output. Every data-extraction pipeline in T5b ships with one.

Fine-Tuning for Economics

From general representations to your domain, at single-GPU cost

Fine-Tuning for Economics

- A pretrained LLM has *general* linguistic knowledge. Economic tasks often benefit from a small, targeted update.
- **Three common strategies:**
 - **Full fine-tuning:** update every parameter. Highest quality, highest cost. Risks overwriting general knowledge with limited task data (catastrophic forgetting below $\sim 1,000$ labels).
 - **Adapters / LoRA** (Low-Rank Adaptation): freeze the original weight W and learn a low-rank update AB^T . Trains $\sim 0.1\%$ of the parameters at near full-FT quality. *Preferred* for most economics applications.
 - **Domain Adaptive Pretraining (DAPT):** continue pretraining on a domain corpus (FOMC statements, MD&A texts, central bank speeches) before fine-tuning on the labeled task.
- **Examples:** sentiment classification on FOMC statements, 10-K segment labeling, NER for corporate filings, summarization of monetary policy reports.

Corpus choice is a validity decision, not a tuning trick. Scoring “financing pressure” in Chinese MD&A with generic BERT-Chinese vs. after DAPT on five years of filings is not just an accuracy difference: the generic model’s confusion over disclosure jargon (“liquidity management”, “refinancing risk”) becomes measurement error in $m(x_i)$ attenuating $\hat{\beta}$ exactly like a badly chosen proxy^{12/28}

Why LoRA Works: The Low-Rank Intuition

- **The empirical fact** (Hu et al. 2021, Fig. 3): the task-specific weight update $\Delta W = W_{\text{fine-tuned}} - W_{\text{pretrained}}$ is *nearly low-rank*. Most of the singular-value mass lives in ≤ 10 directions.
- **Economic analogy.** The cross-sectional variation in FOMC sentence stance is not 7B-dimensional. It lives on a small number of latent dimensions: hawk–dove, topic (inflation vs. employment), tense (description vs. forecast), and a handful of speaker idioms. *The fine-tuning task lives in a low-rank subspace; the parameter update should too.*
- **When rank r matters:**
 - *Rank 4–8 suffices* for stance/sentiment classification, NER, paraphrasing — tasks that re-weight existing knowledge.
 - *Rank 16–64* for instruction following, multi-turn dialogue — tasks that compose existing skills in new ways.
 - *LoRA fails* when the task demands broad lexical change (e.g. adapting a generic LLM to medieval Latin) or memorizing new factual associations — those need full DAPT or RAG, not a low-rank patch.
- **Why this matters for empirical economists.** The compute cost of fine-tuning a 70B model on

Fine-Tuning: LoRA Mechanics

- **Idea:** instead of updating every weight matrix W in the model, freeze W and learn a *low-rank correction*:

$$W_{\text{eff}} = W + AB^{\top}, \quad A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{k \times r}, \quad r \ll \min(d, k).$$

Only A and B are trainable during fine-tuning; the original W stays fixed.

- **Parameter savings** (LLaMA-7B):
 - Full fine-tuning updates $\sim 7 \times 10^9$ parameters.
 - LoRA with $r = 8$ on attention projections updates $\sim 4 \times 10^6$ parameters ($\approx 0.06\%$ of the model).
 - Near-equivalent task quality for most classification, sentiment, and style tasks.
- **Why it matters for economists:** a 7B or even 70B base model can be adapted on a single GPU or a modest cloud instance — no industrial-scale hardware required.
- **Inference:** after training, merge the update back into the weights ($W \leftarrow W + AB^{\top}$); deployment cost is identical to the base model. The adapter itself is tens of MB — swap it per task, version it, share it.
- **Tooling:** HuggingFace `peft` wraps any pretrained model with LoRA adapters in a few lines of code.

Fine-Tuning: Workflow for an Economics Task

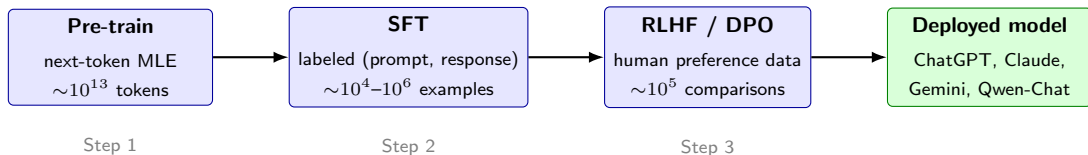
Running example: classify FOMC statements as hawkish / dovish / neutral.

- **Dataset:** 500–2,000 hand-coded statements (e.g. Hansen–Kazinnik, 2023). Split by *meeting date* — train on 1994–2015, validate on 2016–2019, test on 2020–2024 — to avoid leaking future regimes into training.
- **Base model:** an encoder (BERT / RoBERTa) with a classification head is usually enough and cheaper than decoder fine-tuning for three-way classification.
- **Task formatting:** prepend [CLS], tokenize and pad to a fixed length, attach a softmax head over 3 classes to the [CLS] output vector.
- **Hyperparameters (LoRA defaults):** learning rate $1\text{--}3 \times 10^{-4}$, rank $r = 8$, batch size 16–32, 3–5 epochs, early stopping on validation macro-F1.
- **Evaluation:** report macro-F1 and a per-class confusion matrix; hand-audit 20–30 misclassifications — most errors reveal label noise or systematic biases worth fixing before scaling.
- **Reference template:** CentralBankRoBERTa (T5b) follows exactly this recipe, with a DAPT pass on the FOMC corpus preceding the labeled fine-tuning.

Alignment

From “completes text” to “answers questions” — and what that does to measurement

From Base Model to Assistant: Three Stages



- **Pre-training** alone gives a model that completes text. It does *not* answer questions or follow instructions reliably — ask a base model a question and its most likely continuation is *another phrasing of the question*.
- **SFT + RLHF** is what turns a base model into a ChatGPT-style assistant.
- Each stage uses progressively less data but more targeted human effort; step 1 consumes $>90\%$ of the compute, steps 2–3 determine *how the model answers*.

Supervised Fine-Tuning (SFT) and Instruction Tuning

- **Setup:** hand-curated dataset of (instruction, ideal response) pairs — often 10,000–1,000,000 examples.
- **Loss:** same next-token cross-entropy as pre-training, but only on the response tokens, conditional on the instruction.

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{(x,y)} \sum_{t=1}^{|y|} \log P_{\theta}(y_t \mid x, y_{<t}).$$

- **What this does:**
 - Teaches format: “when asked a question, return an answer, not a continuation.”
 - Teaches refusal patterns and helpfulness conventions.
 - Often combined with diverse instruction collections (FLAN, Alpaca, ShareGPT).
- **What it cannot do:** each instruction carries *one* demonstration answer, so the model never learns which of two decent answers is *better* — that gap is what preference learning (next slide) fills.
- **Connection to economics fine-tuning above:** full FT and LoRA from earlier slides are also used here — SFT is just fine-tuning with a particular kind of data (instruction-response pairs)

RLHF: Aligning to Human Preferences

- **Reinforcement Learning from Human Feedback** (Christiano et al., 2017; Ouyang et al., 2022):

1. For each prompt, sample two responses y_A, y_B from the SFT model.
2. A human labels which is better: $y_A \succ y_B$.
3. Train a **reward model** $r_\phi(x, y)$ on these comparisons (Bradley–Terry):

$$P(y_A \succ y_B \mid x) = \sigma(r_\phi(x, y_A) - r_\phi(x, y_B)).$$

4. Fine-tune the LLM with **PPO** to maximize $\mathbb{E}[r_\phi(x, y)]$ subject to a KL penalty against the SFT policy:

$$\max_{\theta} \mathbb{E}_{x, y \sim \pi_{\theta}} [r_\phi(x, y)] - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{SFT}}).$$

- **The KL penalty is essential:** without it, the policy over-optimizes the reward model and “hacks” it (Goodhart’s law in action — repetitive phrasing, over-apologizing).
- **DPO** (Rafailov et al. 2023) skips the reward model entirely: a supervised pairwise loss on the same preference data — no PPO rollouts, much cheaper, numerically stable. Most open-weights chat models (LLaMA-Instruct, Qwen-Chat, Mistral-Instruct) now ship with DPO or a variant (IPO, KTO, ORPO). The algorithm moves fast; the goal does not.

Why Alignment Matters: Helpful, Honest, Harmless

- Anthropic's "HHH" framing (Askell et al. 2021) is the standard rubric:
 - **Helpful**: actually solves the user's task.
 - **Honest**: does not assert things it has no basis for; expresses calibrated uncertainty.
 - **Harmless**: refuses to produce content that materially harms others.
- **Tensions are real**:
 - Helpfulness can pull against harmlessness (over-refusal vs. over-compliance).
 - Honesty can pull against helpfulness ("I don't know" is honest but may be unhelpful).
 - Alignment is a multi-objective trade-off, not a scalar.
- **For research use**:
 - Aligned models are easier to use for first-pass tasks.
 - But alignment can suppress edge-case completions you might want (e.g. negative sentiment templates). Sometimes a base model is preferable for measurement.
 - Constitutional AI (Bai et al. 2022): replace human comparisons with AI-generated critiques against a written constitution — scales the alignment budget.

Alignment's Measurement Consequence: The Version Is the Instrument

What preference tuning adds, preference tuning can silently change.

- SFT demonstrations and preference data are *proprietary, rolling* assets. From outside, all you observe is a version string: gpt-4-0613 vs. gpt-4-2024-08-06.
- Two versions can score the same FOMC sentence differently — not “smarter” or “dumber”, just a different internal prior about what a good answer looks like.
- Evidence: GPT-4's zero-shot hawk-dove labels agree with expert coders *better than fine-tuned encoders do* — and shift with prompt wording and API version (Hansen & Kazinnik 2023). The capability and the instability arrive together.
- Minimum disclosure whenever an aligned LLM produces a variable: provider, model name, exact version, call date, sampling parameters — and archive the raw outputs. (T5b turns this into a full workflow.)

An aligned LLM used for scoring is a **time-varying measurement instrument**. Version-pin it like lab equipment, and treat a version change as an instrument change — because it is one.

Scale

Power laws, compute-optimal training, and the emergence debate

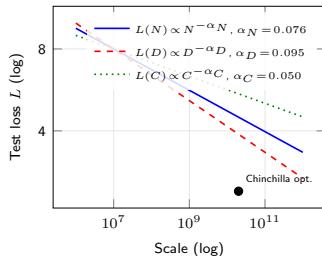
Kaplan Scaling Laws (2020): Loss is a Power Law

- Kaplan et al. (2020) trained dozens of GPT-style models varying N (params), D (data), C (compute).
- **Test loss falls as a power law in each:**

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D},$$

with $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$.

- **Why this is striking:** no plateau over 7 orders of magnitude. Loss keeps falling smoothly with scale.
- **Implication:** for fixed compute $C \sim 6ND$, you can predict optimal (N, D) allocation *before* training.
- **For an economist:** a production function for capability — parameters and data as



Three power-laws on one set of axes: parameters N , data D , compute C . Chinchilla optimum is where the N -curve and the D -curve cross under a fixed compute budget.

Chinchilla (Hoffmann et al. 2022): $\text{Data} \propto \text{Params}$

- Kaplan suggested “mostly scale parameters.” Chinchilla revisited the question with a different fitting method and found:

$$\text{compute-optimal} \implies D^* \approx 20 \cdot N^*.$$

For each parameter, you want ~ 20 training tokens.

- **Earlier models were undertrained.** GPT-3 (175B params, 300B tokens) used ~ 1.7 tokens/param. A Chinchilla-optimal model at 175B would need $\sim 3.5\text{T}$ tokens.
- **Chinchilla itself:** 70B params trained on 1.4T tokens — beat 280B Gopher on most benchmarks.
- **Modern overtraining:** for inference cost reasons, frontier models now go *past* Chinchilla-optimal — e.g. Llama-3 8B trained on 15T tokens ($\sim 1,900$ tokens/param). Smaller, more-overtrained model = cheaper to deploy.
- **The economic frame:** pre-train compute is a one-time fixed cost; inference compute is a recurring marginal cost. Optimal allocation depends on expected serving volume — a marginal-vs-fixed-cost substitution any economist will recognize.

Emergent Capabilities — and the “Mirage” Critique

- **Wei et al. (2022):** on certain tasks (multi-step arithmetic, BIG-Bench, in-context learning), accuracy is near random for small models and then *jumps* sharply at a critical scale.
- Examples cited: 3-digit addition, word-in-context disambiguation, instruction-following.
- Coined the term *emergent capabilities*: not present in smaller siblings, suddenly present in larger ones.
- **Schaeffer, Miranda & Koyejo (2023, “Are Emergent Abilities a Mirage?”):**
 - The sharp jumps may be artifacts of *discontinuous metrics* (exact-match accuracy).
 - Switch to a continuous metric (e.g. token edit distance, partial credit) and the curves become smooth power-laws.
 - “Emergence” may be in our metrics, not in the underlying loss.
- **Where this leaves us:**
 - Capability *does* grow with scale — empirically robust.
 - Whether it grows “smoothly” or “in jumps” depends on how you measure.
 - For an economist: never infer “the model can do my annotation task” from a parameter count — test it *on that task*, against a gold standard.

The Open-Weights Menu — and Where It Runs

For confidential text, the model comes to the data — not the reverse.

Model	Type	L	d	Params	Pretraining corpus
BERT-Base	encoder	12	768	110M	books + Wikipedia (16GB)
RoBERTa	encoder	24	1024	355M	160GB multi-source
BERT-wwm-ext (HIT)	encoder	12	768	110M	Chinese wiki + Baidu Baike + news
GPT-3	decoder	96	12288	175B	filtered web (570GB)
Qwen-7B	decoder	32	4096	7B	Chinese-heavy, ~3T tokens
DeepSeek-V3	decoder (MoE)	61	7168	671B (37B active)	~14.8T tokens, zh+en

- **Compliance driver (China):** the Personal Information Protection Law (PIPL, 2021) and Data Security Law regulate cross-border transfer of personal data — sending firm filings with personal information or survey text to an offshore API is a legal exposure. Locally deployed Qwen / DeepSeek / ChatGLM / LLaMA is the audit-safe default.
- Long-context variants (128k tokens) now cover complete annual reports; MoE models (DeepSeek-V3: 671B total, 37B active per token) infer at mid-size dense cost.
- Same logic elsewhere: IRB-restricted transcripts, proprietary bank data, unpublished co-author drafts — local open weights or explicit data-handling sign-off.

Synthesis

Old friends, new engine — and the one equation, revisited

Transformer vs. LDA Topic Models

LDA / topic models

- Document = mixture of latent topics
- Bag of words: ignores order
- Output: topic-word and document-topic distributions
- Strength: interpretable themes
- Weakness: cannot generate coherent prose

Transformers

- Token-by-token, order-aware
- Captures syntax and long-range dependencies
- Output: contextual embeddings + a generative model
- Strength: rich generation, fluent prose, classification
- Weakness: less directly interpretable

When to use what: LDA is still excellent for thematic structure and quick descriptive analysis on small corpora (it powered Hansen–McMahon–Prat, T1). Transformers dominate when you need generation, classification, or context-aware embeddings.

Three Ways to Adapt an LLM

Method	What changes	Best for
Prompt engineering	Only the input text	Quick experiments, zero-cost adaptation
Fine-tuning (LoRA / DAPT)	Some model weights	New behavior or style on a fixed domain
Retrieval-Augmented Generation (RAG)	Relevant external context injected at inference time	New <i>facts</i> the model never saw

- All three are non-exclusive. In practice you often combine them.
- RAG addresses the limit we meet head-on in Act 5: *the model's knowledge has a cutoff date*.
- Lecture 8 is dedicated to RAG.

Revisiting: Why One Equation Enables So Much

- Now that we understand pretraining, fine-tuning, and generation, T1's opening claim makes mechanical sense:

$$p(t^{(n+1)} \mid t^{(1)}, \dots, t^{(n)}) \quad \text{is all there is.}$$

- **Summarizing a 10-K filing:** the prompt carries the document and the instruction; the model predicts summary tokens one by one because pretraining exposed it to millions of (long-document, summary) pairs.
- **Extracting FOMC stance:** fine-tuning on labeled central bank text shifts the next-token distribution so that stance-label tokens are most probable after a monetary-policy sentence.
- **Generating scenario forecasts:** condition on macro variables and a scenario description; next-token prediction fills in the narrative consistently with the conditioning.
- **Writing and debugging code:** the training corpus included billions of code tokens; the objective is identical — next token, given context.
- **The unifying insight:** all tasks are the same computation. Pretraining provides the knowledge; prompts and fine-tuning steer *which* next-token distribution we sample from. Every capability is a special case of the one equation.

Bridge: A Capable Model Is Still a Fallible Instrument

Everything an LLM does is the one equation — including its failures.

- The same next-token engine that summarizes a 10-K also *hallucinates* plausible fictions, drifts across API versions, and loses facts buried mid-context.
- **Act 5 turns the lens around:** first the five structural limits and what each does to research (T5a); then what embeddings and validated LLM calls can already measure — and the workflow that makes the measure defensible (T5b).

*Arc: T1 FRAME → T2 REPRESENT → T3 ATTEND → **T4 PRETRAIN** → T5 MEASURE.*

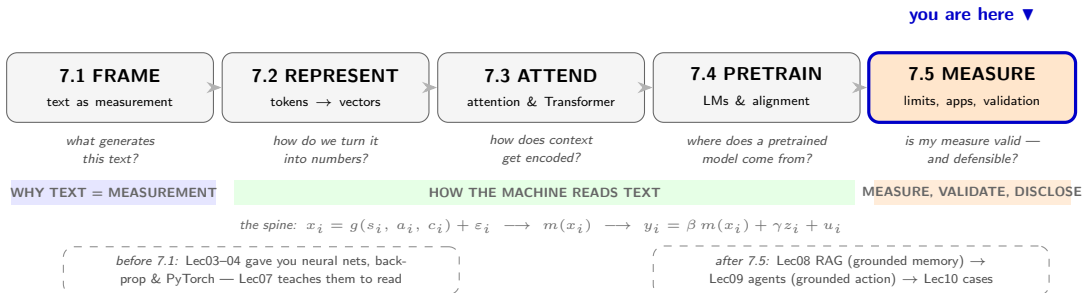
AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 7.5a: Limits of LLMs

Zhigang Feng

2026-07-07 05:00:30

Lecture 7 in One Map



Route: five structural limits, each traced from its architectural root to its research implication (part 1/2 of MEASURE — what breaks); part 2/2 covers what works, and how to defend it.

Five Structural Limits

Architecture in, research implication out

Five Limits, Five Architectural Roots

None of these are bugs awaiting a patch — each follows from how the machine is built.

Limit	Architectural root	Research implication
1. Knowledge cutoff	knowledge frozen in weights; updating = retraining	provide the sources in the prompt; never assume “it knows the event”
2. Hallucination	max-likelihood over <i>form</i> , not truth	every citation and number is a draft until independently verified
3. Context window	attention cost $O(n^2)$	chunking severs cross-references; full filings may not fit
4. Lost in the middle	positional attention bias (U-curve)	extraction accuracy depends on <i>where</i> the passage sits — non-classical error
5. Prompt sensitivity & primacy	ICL: the prompt is the prior	freeze, version, and publish prompts; randomize option order

The next five frames take each limit in turn; the capstone shows them compounding into a 3.6× measurement gap across annotator models.

Limit 1 — The Knowledge Cutoff

- Every pretrained LLM is trained on a *static snapshot* of text. After the snapshot ends, the model learns nothing new.
- Each model has a **knowledge cutoff date**. After this date:
 - Events are invisible to the model
 - New papers, datasets, policy announcements simply don't exist as far as it is concerned
 - Asking “what happened in March 2026?” against a model trained through 2024 will at best yield “I don't know” — and at worst, a confident fabrication
- **Why not just retrain?**
 - Cost: \$10M–\$100M+ for a frontier model
 - Time: weeks to months
 - Energy and infrastructure constraints
 - Practically impossible to keep “current”
- This is not a bug — it is structural: the knowledge lives *in the weights*. Any solution that needs *fresh* information must work *around* the cutoff, not against it. **This is the core motivation for Lecture 8.**

Limit 2 — Hallucination

What it looks like

- Fabricated citations to papers that do not exist
- Made-up statistics that look plausible
- Non-existent laws, cases, article numbers
- Subtly wrong details in otherwise correct prose

Mitigations

- Require and verify citations externally
- Use retrieval (Lec08) to ground answers in real documents
- Use tools / code: numbers from a calculator
- Validate a sample by hand
- Set temperature $\tau = 0$ when measuring

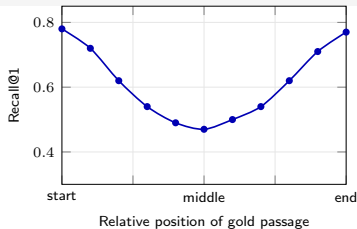
Why does it happen? The training objective (T4) maximizes the likelihood of *plausible-sounding* next tokens, not *true* ones. Where training data is sparse, the model fills the gap with the right *form* and invented *content*. Truth is a property of the world; plausibility is a property of language.

Limit 3 — Context Window and the $O(n^2)$ Wall

- Even modern models with 100,000+ token context cannot swallow arbitrarily large datasets:
 - A 200-page 10-K filing easily exceeds short context windows
 - Decades of FOMC transcripts cannot be loaded at once
 - Multi-firm or multi-country panels of textual data certainly cannot
- **Practical consequences:**
 - Long inputs must be *chunked*, which can break coherence and cross-reference — MD&A's reference to a risk factor stated 30 pages earlier is severed by the chunk boundary
 - Important information may sit outside the window altogether
 - Attention cost is $O(n^2)$ in the context length, so doubling n quadruples compute and memory
- Newer architectures (sparse attention, sliding-window attention, RoPE-extension, state-space hybrids) push the limit but do not eliminate it — and they trade away exact global attention (T3b).
- **Implication:** you cannot just dump “all the data” into a prompt. You need a way to *retrieve* the relevant pieces and feed them in — another argument for RAG.

Limit 4 — Lost in the Middle: Positional Saliency

- **Liu, Lin, Hewitt, Paranjape, Bevilacqua, Petroni, Liang** (TACL 2024): when the relevant fact sits in the *middle* of a long prompt, recall drops 20–30 percentage points relative to its position at the start or end.
- **U-shaped recall curve.** Across closed (GPT-3.5/4) and open (LLaMA-2, MPT) LLMs, accuracy is high when the gold passage is near the start or the end of the context window and sags in the middle — a positional bias in attention, not random noise.
- **Two implications:**
 - *Stuffing the prompt* (concatenate as much as it fits) is **not a substitute for retrieval**: padding adds noise that buries the signal.
 - RAG (Lec08) must therefore *rank* chunks, not merely retrieve them. Top-ranked chunks should be placed at the prompt's ends.



Stylized U-shape from Liu et al. (2024); same qualitative pattern across model families.

Limit 5 — Prompt Sensitivity and the Primacy Effect

Generic prompt:

Classify this sentence:
"Inflation remains elevated and the Committee remains highly attentive to inflation risks."
Label:

Model output: *neutral* — the descriptive verbs ("remains") and the absence of a directional action word dominate.

Role-conditioned prompt:

You are a monetary economist. Is the following sentence hawkish, dovish, or neutral?
"Inflation remains elevated and the Committee remains highly attentive to inflation risks."
Label:

Model output: *hawkish* — "highly attentive to inflation risks" is read as a tightening signal once the prior is monetary policy.

- Both prompts pass the same sentence to the same model. The difference is the *prior* the role + label space imposes (cf. ICL as Bayesian inference, T4).
- **Brucks & Toubia (2025, PLOS ONE)**: primacy bias on balanced binary tasks — GPT-4 frequently picks the *first* option (a strong, robust primacy bias). Randomize option order; average across runs.
- **WEIRD bias & version drift**: training corpus is Western/English-leaning; closed-source models change silently between API calls (T4's "version = instrument").
- **Take-away**. Treat the prompt as part of the research design — freeze it, version it, report it. Robust analyses report results under multiple paraphrases.

Capstone: When the Limits Compound — YVP (2026)

Three frontier models, one frozen prompt, 18,797 O*NET tasks — and a $3.6\times$ gap.

- Yin, Vu & Persico (NBER WP 35110) re-run the standard AI-exposure rubric (Eloundou et al.) with three 2025–26 frontier models — everything else held fixed.
- Mean exposure differs $3.6\times$ across annotator models; pairwise agreement $\approx 57\%$, Cohen's κ as low as **0.36** (hand-coding publication standard: $\kappa \geq 0.75$).
- Downstream, the same DiD regression on each model's scores: individual-level coefficients vary $2.4\times$; the county-level estimate **flips sign** (significant negative $\rightarrow$ insignificant positive).

Model version = measurement instrument. Pin it, disclose it, and validate against a human gold standard — or the regression inherits whichever model you happened to call. T5b performs the full autopsy of this literature and builds the defensive workflow.

Discussion: which of the five limits drives the divergence? Mostly #5 — prompt/prior sensitivity compounded by each vendor's alignment choices (T4) — with #2 contributing at the rubric boundaries.

The Brain Metaphor

An LLM is like a very capable brain that finished school on a fixed date.

What the brain knows

- General language and reasoning
- Broad economics, math, programming
- Published papers up to its cutoff
- Common patterns, idioms, code recipes

What the brain does not know

- Anything that happened after the cutoff
- Your private FOMC corpus
- A specific firm's internal documents
- This morning's data release
- Your co-author's unpublished draft

Question: how do we let this brain access the information on the right *without* retraining it? (Lecture 8's opening question — and T5b's closing bridge.)

Bridge: Knowing the Limits, Use the Strengths

The audit is done; now the toolkit.

- Nothing in this deck says “don’t use LLMs.” It says *unvalidated* use is what fails — and each limit has a research-design counter: ground it, pin it, audit it, randomize it.
- **T5b**: what embeddings already measure (similarity, classification, networks); the case-study record from ECB Word2Prices to asset embeddings; the AI-exposure cautionary tale in full; and the six-step validation workflow — ending on the lecture’s thesis.

Arc: T1 FRAME → T2 REPRESENT → T3 ATTEND → T4 PRETRAIN → T5 MEASURE.

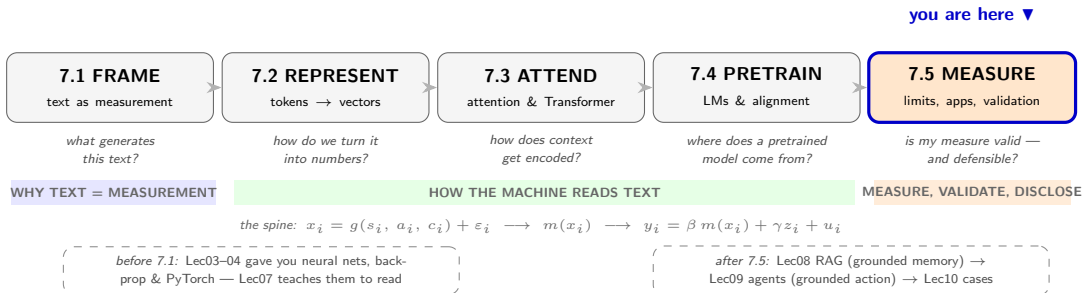
AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 7.5b: Applications and Validation

Zhigang Feng

2026-07-07 05:00:38

Lecture 7 in One Map



Route: three featured cases → the 2024–26 frontier → embeddings as regression variables → the AI-exposure cautionary tale → the validation workflow → the thesis (part 2/2 of MEASURE, and the close of Lecture 7).

Three Featured Cases

Word2Vec forecasting, TF-IDF exposure, domain encoders — Thread A and B pay off

Three Featured Case Studies

1. **ECB Word2Prices** (Araujo, Bokan, Comazzi & Lenza, 2025)
 - Word2Vec on ECB statements + BVAR for inflation forecasting
2. **Patents and Worker Exposure** (Kogan et al., 2019)
 - TF-IDF + cosine similarity between patents and occupational tasks
3. **Central Bank Sentiment with BERT** (Pfeifer & Marohl, 2023; Ganguli et al., 2024)
 - Fine-tuned RoBERTa for sentiment toward different economic agents
 - PaECTER patent embeddings for innovation dynamics

After the three deep dives: the 2024–26 frontier, then the tools that turn embeddings into regression variables — and the validation discipline that makes any of it publishable.

Case 1 — ECB Word2Prices: Setup

Araujo, Bokan, Comazzi & Lenza (2025), ECB WP No. 3047

- **Question:** can embeddings of ECB communications improve euro-area inflation forecasting?
- **Data:**
 - ECB press conference introductory statements, 2002Q1–2023Q4
 - Target: HICP_x (core inflation), monthly → quarterly
- **Method:**
 - For each quarter q , retrain Word2Vec on the corpus available *up to* q (no look-ahead)
 - Average word vectors per statement; aggregate to a quarterly text vector
 - Stack text vector with macro variables; fit a Bayesian VAR
 - Forecast 1–4 quarters ahead
- **Why this design?** Re-training quarterly avoids the look-ahead bias that plagues most LLM-based forecast exercises (T2a's caution, obeyed to the letter), while remaining computationally cheap.

Case 1 — Python Skeleton

```
# Quarterly Word2Vec retraining for ECB inflation forecasting
from gensim.models import Word2Vec
from statsmodels.tsa.api import VAR
import numpy as np

forecasts = []
for q in quarters:
    # 1. Build corpus available at quarter q (no look-ahead)
    corpus_q = [s for s in all_statements if s.date <= q]
    # 2. Train Word2Vec on the available corpus
    w2v = Word2Vec(corpus_q, vector_size=100, window=5,
                   sg=0, min_count=2, epochs=20)
    # 3. Average word vectors per statement, then per quarter
    stmt_vec = [np.mean([w2v.wv[w] for w in s if w in w2v.wv], axis=0)
                 for s in corpus_q]
    text_q = aggregate_quarterly(stmt_vec, dates)
    # 4. Stack with macro variables and fit a BVAR
    Y = np.column_stack([macro_q, text_q])
    bvar = VAR(Y).fit(maxlags=4)
    # 5. Forecast 1-4 quarters ahead
    forecasts.append(bvar.forecast(Y, steps=4))
```

Case 1 — Results and Lessons

- **Forecast performance:**

- BVAR with Word2Vec text features outperforms a baseline AR / BVAR without text at horizons of 1–4 quarters
- Sentiment dictionaries (e.g. Loughran-McDonald) deliver only marginal gains relative to the no-text baseline
- Heavy LLMs (BERT, fine-tuned) can match or beat Word2Vec but are much more expensive and harder to retrain quarterly — look-ahead bias becomes a real risk

- **Take-aways for empirical economists:**

- “Lighter is better” if it lets you respect the timing structure of the forecasting exercise
 - Frequently-retrained Word2Vec is a strong baseline before reaching for an LLM
 - Always document: training corpus cutoff at every step, vector dimension, window, retraining frequency
- Replicable framework with zero GPU requirements — ideal for grad-student empirical work.

Case 2 — Patent Text and Worker Exposure

Kogan, Papanikolaou, Schmidt, Seegmiller (2019, 2023)

- **Question:** how does exposure to technological innovation affect workers' earnings and employment?
- **Method:**
 - Compute TF-IDF vectors for U.S. patent texts and for O*NET occupational task descriptions
 - Measure cosine similarity between each occupation and each patent
 - Aggregate up: occupation-level technological exposure
- **Findings:**
 - High-exposure occupations experience earnings declines and reduced employment stability
 - Effects are present for both low- and high-wage workers
- **Why this is in our lecture:** a textbook example of how a *simple* method (TF-IDF + cosine, T2a's toolkit) on a *thoughtfully chosen* corpus can answer a substantive economic question. You don't need transformers to make a contribution, but you do need a clean identification strategy.

Why Generic LLMs Underperform on Financial Text

The vocabulary mismatch. Generic BERT/GPT was trained on Wikipedia, web text, and books — not on disclosure language.

- **Polarity flips.** “Liability,” “aggressive,” “exposure” are negative in everyday English but neutral or positive in accounting/finance. Loughran & McDonald (2011) is the canonical correction of the older Harvard-IV-4 list.
- **Hedging language.** “May,” “could,” “intend to” are stop-words in generic NLP but encode legal disclosure stance in 10-K text.
- **Numbers in context.** Subword tokenizers split 1.2345 into digit tokens; without finance fine-tuning the model treats the magnitude as just another vocabulary item.

Two responses: (1) train a domain-specific encoder (next slide), or (2) zero-shot prompt a frontier LLM and validate carefully (Hansen & Kazinnik, 2023).

Case 3a — FinBERT and CentralBankRoBERTa: Two Domain Encoders

	FinBERT (Araci, 2019; Yang et al., 2020)	CentralBankRoBERTa (Pfeifer & Marohl, 2023)
Backbone	BERT-base (12 layers, 110M params)	RoBERTa-base, fine-tuned
Pre-train corpus	Reuters TRC2-Financial + 10-K corpora	RoBERTa-base fine-tuned on ~12k hand-labeled sentences (plus ECB/BIS pseudo-labels)
Task it ships for	3-way sentiment (positive/negative/neutral) on financial sentences	Agent-specific sentiment: stance toward five macro agents (households, firms, financial sector, government, central bank)
Headline result	Beats LSTM/dictionary baselines on Financial PhraseBank	Agent-specific scores predict regional inflation & employment better than “hawkish/dovish”
When to reach for it	Fast firm-level sentiment at scale	Central-bank communication studies

Both are fine-tuned BERTs — the architecture is Act 3; the pretraining/fine-tuning recipe is Act 4 (base → DAPT on domain text → small labeled head). This deck cares only about *using* them: load weights, feed sentences, get scores.

Domain Models vs. Frontier LLMs: No Single Winner

Hansen & Kazinnik (2023): “GPT-4 reads Fedspeak.” Zero-shot GPT-4 classifies FOMC sentence stance at expert-level accuracy; beats fine-tuned BERT/RoBERTa; provides human-consistent explanations.

Gambacorta et al. (BIS WP 1215, 2024): CB-LMs. Domain-adapted encoder beats generic BERT on FOMC stance; GPT-4 still wins on complex narrative tasks.

Practical decision rule:

- **Sentence-level sentiment at scale, fixed taxonomy** → FinBERT / CentralBankRoBERTa (cheap, deterministic, citable).
- **Open-ended interpretation, narrative, novel category** → frontier LLM (GPT-4, Claude) with a logged prompt.
- **In doubt** → run both and report agreement (Cohen's κ).

No single model dominates — the validation workflow (later in this deck) handles the choice. And recall T4: the frontier LLM's scores move with its version; the frozen encoder's do not.

Case 3b — PaECTER for Patent Innovation

Ganguli, Lin & Reynolds (2024), NBER WP

- **Goal:** measure innovation dynamics from patent text using transformer-based embeddings.
- **Model: PaECTER** (Patent Embeddings from Citation-Enhanced Transformer Representations)
 - Initialize from BERT-base
 - Fine-tune on patent corpora
 - Use citation pairs as a supervised signal during fine-tuning (related patents pulled together in embedding space)
- **Findings:**
 - Significantly more accurate similarity judgments than TF-IDF or LDA
 - Distinguishes incremental from radical innovations
 - Long-run trend: patents are increasingly diverging in semantic space (broader exploration)
- **Take-away:** fine-tuned transformer embeddings give a richer, citation-aware view of technological evolution than purely text-based or count-based methods.

The 2024–26 Frontier

Central-bank communication and asset pricing move to LLM-scale

Frontier (2024–25): LLMs for Central Bank Communication

- **CB-LMs** (Gambacorta et al., BIS WP 1215, 2024)
 - Domain-adapted encoder retrained on 37,000 central bank papers + 18,000 speeches
 - Outperforms generic BERT on FOMC stance classification
 - But: GPT-4 wins on complex narrative tasks — no single model dominates
- **GPT-4 for FedSpeak** (Hansen & Kazinnik, 2023)
 - GPT-4 classifies FOMC statement sentences at *expert-level* accuracy
 - Outperforms fine-tuned BERT/RoBERTa; provides human-consistent explanations
 - Zero-shot performance collapses domain-adaptation barrier for many tasks
- **GPT-4o on 25,000 FOMC Sentences** (Federal Reserve FEDS Note, December 2024)
 - Topic classification across 2010–2024 minutes at scale
 - Detects “financial stability” discourse spikes in 2020 and 2023 banking crises
 - LLMs reveal *thematic shifts* that sentiment scores miss
- **74,882 Central Bank Documents** (Silva, Moriya & Veyrune, IMF WP 2025/109)
 - 169 central banks, 1884–2025; LLM-coded stance + directional communication index
 - Forecasts future policy rate changes; documents shift to forward-looking language

Frontier (2024–25): LLMs in Asset Pricing

- **Expected Returns and LLMs** (Chen, Kelly & Xiu, NBER WP 31502, 2024)
 - LLM embeddings of financial news $\Rightarrow$ cross-sectional equity return prediction
 - Substantially outperforms TF-IDF and momentum; gains largest on *negation* and complex narratives — exactly the static-embedding failure cases from T2b
 - Evidence of slow price adjustment to information that only LLM embeddings extract
- **AI Asset Pricing Models** (Kelly, Kuznetsov, Malamud & Xu, NBER WP 33351, 2025)
 - Transformer architecture embedded directly in the stochastic discount factor M_{t+1}
 - Multi-head attention learns cross-asset dependencies from 132 stock characteristics
 - Large reductions in pricing errors vs. previous ML factor models
- **Asset Embeddings** (Gabaix, Koijen, Richmond & Yogo, NBER WP 33651, 2025)
 - Embeddings learned from *investor portfolio holdings* via skip-gram (same math as T2b's Word2Vec)
 - Arithmetic: `tech + growth - hardware` $\approx$ `software firms`
 - GPT-4 generates economic narratives for firm clusters in the learned embedding space
- **Key insight:** the same transformer and embedding toolkit that encodes language can encode financial structure — text and quantitative data are no longer separate worlds.

Embeddings as Regression Variables

Similarity, classification — and the standard errors they deserve

Cosine Similarity as an Economic Variable (Thread B)

The most direct embedding-to-regression path: distance between documents.

- $\text{sim}(i, j) = \frac{\mathbf{e}_i^\top \mathbf{e}_j}{\|\mathbf{e}_i\| \|\mathbf{e}_j\|}$; cosine *distance* = $1 - \text{sim}$. Length-invariant by construction — and that matters, because document length tracks firm size and disclosure habits.
- **Statement-change series:** $\delta_t = 1 - \text{sim}(D_{t-1}, D_t)$ across consecutive FOMC statements gives a continuous “how much did the message change” measure — finer than hawk/dove labels. (Similarity tracking of consecutive Fed statements goes back to Acosta & Meade 2015.)
- **Interpretation discipline — two layers of endogeneity:**
 - the *expected* component: strip out what markets already priced, e.g. with high-frequency windows around the release;
 - the *structural* component: the bank may deliberately say more when conditions worsen — δ_t then responds to the economy rather than shocking it. No embedding quality fixes that; only a design can.

The DGP one last time: δ_t is an $m(x)$. Whether it is a *shock* or an *outcome* is an identification claim about a_t and c_t — not a property of the vector space.

Validation Caveats for Econometricians

- **Cosine similarity is not innocent.** Three common confounds inflate similarity between firm/policy texts:
 - **Industry jargon:** firms in the same SIC code share boilerplate language
 - **Templates:** regulatory filings re-use chunks of text verbatim
 - **Length:** longer documents have smoother averages and look more similar
- **Standard tools for robustness:**
 - Permutation tests — shuffle labels within an industry, check if observed similarity exceeds the within-industry baseline
 - Residual embeddings — subtract the industry mean before measuring similarity
 - Bootstrap the entire pipeline (text $\rightarrow$ embedding $\rightarrow$ measurement $\rightarrow$ regression) for honest standard errors (next slide)
- **Dictionary methods are not innocent either.** The Loughran-McDonald (2011) finance-specific dictionary corrects mismatches in the older Harvard IV-4 list (e.g. “liability” is not negative in financial accounting).
- **Always validate** a sample of model outputs against human coding before scaling up.

The Generated-Regressor Problem

Your regressor was estimated — the second stage must know that.

- An embedding similarity, a topic share, an LLM score: all are *estimates*, carrying error from model choice, training data, and aggregation.
- OLS with a noisy constructed regressor $\Rightarrow$ standard errors too small, t -statistics too big — the classic two-step problem, now wearing NLP clothes.
- Worst when the first stage is trained on little data: embeddings fit on a few hundred FOMC statements carry large estimation error into every downstream δ_t .
- **Fix: bootstrap the whole pipeline** — resample documents, re-embed, re-aggregate, re-run the regression — so first-stage uncertainty propagates into the reported intervals. Report pipeline-bootstrap intervals next to the naive ones.

Rule of thumb: if deleting one year of documents would visibly move your embedding space, then second-stage standard errors without the full-pipeline bootstrap are overconfident.

Zero-Shot Classification: Measuring Concepts with No Training Labels

Describe the class in a sentence; let the geometry do the rest.

- $\hat{c} = \arg \max_{c_k} \text{sim}(\mathbf{e}_{\text{doc}}, \mathbf{e}_{c_k})$: embed the document and each class *description* (“this firm faces material climate-transition risk”), assign by nearest neighbor.
- **Why it matters for economics**: new constructs — climate exposure, AI adoption, supply-chain geopolitical risk — have *no historical labels*. Zero-shot lets you score decades of filings *backwards* with today’s concept, extending Hassan-style risk measures to categories nobody coded at the time.
- **Model choice is make-or-break**: raw BERT embeddings are anisotropic — unrelated sentences look similar because they share frequent words. Use sentence-embedding models trained contrastively for similarity (SBERT via sentence-transformers, BGE, E5) — T3b’s token-vs-document distinction doing real work.
- **Validation ladder — all three steps, in order**:
 1. consistency across paraphrased class descriptions (Fleiss’ $\kappa > 0.6$);
 2. accuracy on a hand-coded gold subsample — consistency is *not* correctness: a systematically wrong description can be perfectly consistent;
 3. external validity against a real economic outcome.

A Cautionary Tale

The AI-exposure literature — when the instrument is the thing it measures

The Literature's Question: How Exposed Is Each Job to AI?

- **The measurement problem:** map $\sim 19,000$ O*NET tasks (or ~ 900 SOC occupations) to a numeric *AI exposure* score that can be regressed on wages, employment, and county outcomes.
- **Two pre-LLM generations of measures:**
 1. **Felten, Raj, Seamans (2018, 2021)** — AI Occupational Exposure index (AIOE). Link AI benchmark progress (image recognition, translation, ...) to O*NET ability requirements. *Mechanical mapping; human raters set the abilities.*
 2. **Webb (2020)** — text overlap of AI patent abstracts with O*NET task descriptions. *No raters at all — pure TF-IDF.*
- Both treat the measurement instrument as **fixed and inspectable**. A re-run next year produces the same numbers.
- **Economist's framing:** exposure is a *measurement-error* question, not a CS question. The next slide shows what changes when the instrument is itself an LLM.

Three Generations of AI-Exposure Instruments — Compared

Same target (occupation-level exposure score), three very different instruments:

	Felten, Raj, Seamans (2018, 2021)	Webb (2020)	Eloundou et al. (2023)
Instrument	Hand-curated map: AI benchmark → O*NET ability	TF-IDF cosine: AI patent abstracts → O*NET tasks	Single LLM call: GPT-4 rates each task on 4-level rubric
Cost per refresh	High (human raters, weeks)	Low (re-tokenize, rerun)	Low per call; high per provider switch
Stability across re-runs	Identical (deterministic mapping)	Identical (deterministic)	Stochastic ; different provider = different score
Validity assumption	Benchmark progress proxies real capability	AI patent text proxies AI capability	The LLM is a calibrated rater
Auditable by economist?	Yes (rater notes)	Yes (read the patent and the task)	Partially (rationale text); not the model weights
Failure mode	Benchmarks lag the frontier	Patent corpus is noisy	Version drift, hallucination, prompt sensitivity

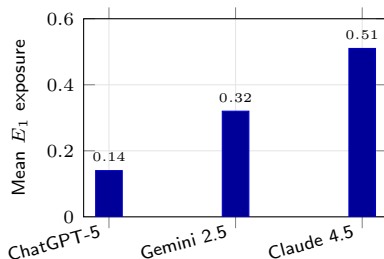
The LLM-as-Rater Turn: Eloundou et al. (2023)

- **Eloundou, Manning, Mishkin, Rock (2023, “GPTs are GPTs”)** — the first paper to use **GPT-4 itself** as the rater on the O*NET task list.
- **Rubric (per task):**
 - **E0** — no LLM speedup.
 - **E1** — direct LLM access reduces task time by $\geq 50\%$.
 - **E2** — speedup achievable with LLM-powered software built atop the model.
- **Procedure:** prompt GPT-4 with a task description, ask it to classify, repeat for $\sim 19k$ tasks, average within occupation $\rightarrow$ exposure score.
- Quickly adopted as the *default* exposure measure — Acemoglu (2024), IMF (2024), BIS, and many follow-ups all rebuild on this LLM-rated input.
- **The instrument is now the same kind of model as the subject of the research** — and as T5a established, that instrument is stochastic, hallucination-prone, and version-drifting.

Replication: Yin, Vu, Persico (NBER 35110, 2026)

Design — only the LLM changes:

- Same Eloundou (2023) prompt and task list (18,797 O*NET tasks, ~ 900 occupations).
- Three frontier 2026 annotators: **ChatGPT-5**, **Claude 4.5**, **Gemini 2.5**.
- Identical decoding (low temperature, deterministic where supported); identical few-shot examples; identical aggregation to occupation-level E_1 means.
- Plug the resulting exposure scores into the same downstream difference-in-differences regressions used by the labor-AI literature.



Same prompt, same tasks — $3.6\times$ divergence in mean exposure.

- Pairwise agreement: **56.9%**, Cohen's κ as low as **0.36**.
- Individual DiD coefficient varies **$2.4\times$** across annotators.
- County-level estimate **flips sign** (significant negative $\rightarrow$ insignificant positive).

What 0.36 means (Landis–Koch benchmarks): < 0.20 slight, $0.21\text{--}0.40$ *fair*, $0.41\text{--}0.60$ moderate, $0.61\text{--}0.80$ substantial, $0.81\text{--}1.00$ almost perfect. Publication-grade hand-coding aims for $\kappa \geq 0.75$ ^{21 / 31}

Attitude in the Literature: LLMs Are Not Static Instruments

- **The error is non-classical:** disagreement concentrates on tasks near the rubric boundary — the high-exposure tasks the regressor cares about. Errors correlate with the regressor.
- Yin et al.: *“the risks of treating evolving LLMs as static instruments.”* A paper rated with GPT-4 in 2023 cannot be re-run in 2026 and get the same labels.
- **Each T5a limit → a measurement consequence:**
 - *Sampling stochasticity* → set $\tau = 0$ and report it.
 - *Hallucination / mis-calibration* → ensemble across models, report Cohen's κ .
 - *Version drift* → pin the snapshot; treat *prompt + model + seed* as one instrument.
- **Emerging best practice:** pre-register prompt + model + decoding; use multiple frontier annotators and report between-model agreement; audit against a hand-coded gold standard; report inter-annotator sensitivity, not a single point estimate.
- **Takeaway:** the LLM is a measurement instrument — calibrate, version, and disclose.

When NOT to Use an LLM Rater — A Decision Rule

Pearson 0.6 from 50 lines of `re.findall` is a real signal. The LLM rater must clear a meaningful bar to justify its cost.

A defensible workflow before reaching for an LLM:

1. **Hand-code 100–200 task gold standard** (you, an RA, two coders for κ).
2. **Fit a simple baseline** — TF-IDF + logistic regression, or a keyword rule with handful of features.
Report κ vs. gold.
3. **Run the LLM in parallel.** Compute $\kappa(\text{LLM}, \text{gold})$ and $\kappa(\text{LLM}, \text{baseline})$.
4. **Decide:**
 - $\kappa(\text{LLM}, \text{gold}) \geq 0.75$ and substantially $> \kappa(\text{baseline}, \text{gold}) \Rightarrow$ LLM is validated; use it, disclose model + prompt.
 - $\kappa(\text{LLM}, \text{gold}) < 0.60$ or $\approx \kappa(\text{baseline}) \Rightarrow$ ship the baseline; LLM doesn't earn the version-drift risk.
 - Between 0.60 and 0.75 $\Rightarrow$ ensemble (LLM + baseline + tie-break by hand on disagreements).

The LLM is one instrument among many, not a default. The Yin–Vu–Persico replication shows what you risk by skipping the audition.

The Application Zoo, Organized by Econometric Role

The same text can play three very different roles. Pick the representation to match the role.

Paper	Signal (regressor $m(x_i)$)	Outcome (behavior y)	Instrument / shock
Baker–Bloom–Davis (2016)	Newspaper keyword EPU $\rightarrow$ VAR	—	EPU surprises
Hoberg & Phillips (2016)	10-K TF-IDF cosine $\rightarrow$ product-market similarity	—	—
Hassan et al. (2019)	Bigram firm-level political risk $\rightarrow$ investment regression	—	Quarterly call shocks
Li et al. (2021)	W2V on earnings calls $\rightarrow$ 5 culture dimensions	—	—
Lopez-Lira & Tang (2023)	ChatGPT zero-shot sentiment on headlines $\rightarrow$ next-day returns	—	—
Hansen–McMahon–Prat (2018)	LDA topics	Committee speech change after transparency reform	1993 reform date
Hartley (2025)	LLM-reconstructed EPU (19th c., OCR)	—	War-news shocks

- *Signal column dominates the literature* — it's the easiest role and the loudest result.
- *Outcome* requires a behavior to be *caused* by something else; harder but cleaner identification.
- *Instrument / shock* use needs the exclusion restriction to survive the prompt/fine-tuning recipe — document it.^{24 / 31}

Validation and Disclosure

The workflow that makes a text measure publishable

A Six-Step Workflow for LLM-Assisted Measurement

1. **Pre-alignment.** Hand-code 100–200 examples; iterate on the prompt or fine-tuning recipe until you reach acceptable agreement.
2. **Full annotation.** Run the model on the entire sample with a frozen prompt and a frozen model version.
3. **Gold-standard audit.** Blind-code 10% of the sample and compute Cohen's κ (publication threshold $\kappa \geq 0.75$).
4. **Category-level reporting.** Don't just report overall accuracy; show the confusion matrix by class. Minority classes are often where models fail silently.
5. **External validity check.** Link the text-derived measure to a real economic outcome (returns, investment, employment) before treating it as a measurement.
6. **Disclosure.** Report prompt, model version, temperature ($\tau = 0$), date, all preprocessing, and provide code. Reproducibility is essential when the model behind the API can change without notice.

Worked Example: Applying the Workflow to FOMC Sentiment

Imagined paper: “GPT-4 sentiment on FOMC statements predicts 10-year yields.”

Step	What you do	What goes in the appendix
1. Pre-align	Hand-code 200 sentences hawk/dove/neutral	Prompt v3 (frozen); κ vs. author = 0.81
2. Annotate	GPT-4-2024-06-13, $\tau = 0$, full corpus	Cost, runtime, retries, sha256 of inputs
3. Gold audit	Blind-code 10% by RA	$\kappa = 0.78$ overall; 0.62 on “neutral”
4. By-class	Confusion matrix	Hawkish recall 0.91; neutral recall 0.66 (flagged)
5. External validity	Predict Δy_{10} at FOMC dates	$\hat{\beta} = 4.2$ bp per unit, $t = 3.1$
6. Disclose	Prompt + code on Zenodo	DOI, model card, replication run-time

LLM-as-judge connection. A second LLM as a cross-checker: same input, different prompt, flag disagreements for human review — the same construction-to-coefficient logic used in the AI-exposure autopsy above, and a preview of Lec09’s sub-agent patterns.

Disclosure Norms for AI-Assisted Empirical Work

Minimum reproducibility package.

- **Model identity.** Provider, model name, exact version string, call date.
- **Prompt(s).** Frozen text in a versioned file; not paraphrased in the paper.
- **Sampling parameters.** Temperature, top- p , max tokens, seed if available.
- **Inputs.** sha256 of every input file; document any preprocessing.
- **Outputs.** Raw model outputs preserved *before* any post-processing.
- **Code.** End-to-end script with retry logic and the exact API client version.

The auditable-call triple: $\tau = 0 + \text{fixed seed} + \text{SHA-256 of every input}$, with raw outputs archived. One reproducible instrument, one line in the appendix.

AEA policy. The October 2024 Data and Code Availability update explicitly covers AI-assisted analysis: report when an LLM was used, for what step, and provide artifacts that let a reader reproduce the result given the same model.

IRB and proprietary text. Survey transcripts, internal memos, and clinical text often cannot be sent to a hosted API. Run an open-weights model locally (T4's menu), or obtain IRB sign-off on the provider's data-handling terms.

The closed-source LLM is the most-changed equipment in your empirical paper. Document it like the dataset it is.

Three Starter Paths for the Macro Researcher

Where to begin, by application — each a full pipeline you can defend.

1. **Central-bank stance.** Domain-adapt an encoder (EconBERT for English; MacBERT + DAPT on PBOC reports for Chinese) → extract [CLS] embeddings → compute δ_t between consecutive statements; benchmark against Romer–Romer narrative shocks as the external anchor; add a LoRA classification head on a small labeled set if you need the hawk/dove *direction*.
2. **Policy uncertainty.** Keep Baker–Bloom–Davis as the transparent, auditable baseline (Thread A's lesson); report a preprocessing specification curve (T2a); test stationarity, and separate *language* breaks from *economic* breaks (T2b) before the VAR.
3. **Nowcasting.** LDA topic shares or embedding principal components as additional factors in the standard nowcast regression — the text block enters exactly like any other factor; but bootstrap the whole text pipeline for the standard errors (this deck).

China parallels ready to run: corporate-culture scores on CNRDS earnings-call text (validate SOE and non-SOE separately — T1); news-based nowcasts on Wind; a PBOC Monetary Policy Report stance series (quarterly since 2001).

The Thesis

What five acts add up to

Identification Precedes Measurement

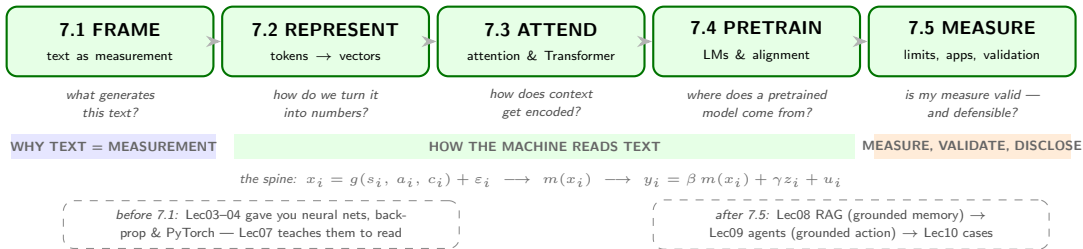
The lecture's thesis: a better text model shrinks measurement error — it never substitutes for a research design.

- Everything in Acts 2–4 improves $m(x_i)$: less noise, more nuance, more recall. *Nothing* in Acts 2–4 makes $m(x_i)$ exogenous.
- The DGP is the reason: strategic framing a_i and institutional context c_i ride along in every representation, sparse or dense, static or contextual.
- Causal claims still come from designs:
 - **DiD** — Hansen–McMahon–Prat's 1993 reform (T1);
 - **IV** — e.g. exogenous CEO turnover (sudden death, health shocks) shifting communication style, with the exclusion restriction argued, not assumed;
 - **RD**, or **narrative identification** (Romer–Romer) as the external anchor for text-based shock series.
- So the first question of a text project is not “which model?” It is “**which parameter, identified how?**” — the model choice then follows from the role the text plays (T1's three roles).

Identification precedes measurement. Text variables serve the design; they do not replace it.

Lecture 7, Complete

all five acts complete



Nine takeaways in one breath: text is a measurement problem, generated by states, framing, and institutions (1); representations evolved sparse $\rightarrow$ dense $\rightarrow$ contextual, each step trading transparency for nuance (2); self-attention is dynamic disambiguation — one vector per occurrence (3); pretrain-then-adapt collapsed the labeled-data requirement (4); alignment made models usable *and* made the version part of the measure (5); five structural limits are architectural facts, not bugs (6); zero-shot classification extends measurement to concepts without labels (7); validation discipline — κ , gold standards, disclosure — is the price of admission (8); and identification precedes measurement (9).

Bridge: From Language Engine to Grounded Systems

Lec07 built the engine; the cure for its parametric weaknesses is grounding.

- **Retrieve** the sources and condition the answer on them — **Lec08 (RAG)**: grounded *memory*. Beats the cutoff and hallucination on *your* corpus; T3b's token-vs-document embeddings are its geometry. (Retraining is too costly and fine-tuning stores style, not facts — retrieval is the practical third path.)
- **Act** through tools that return ground truth — **Lec09 (agents)**: grounded *action*. T4's JSON schemas become tool calls; numbers come from computation, not recollection.
- **The contract either way — cite, then claim**: every factual statement pairs with the retrieved passage or tool output it conditions on. Unpaired claims are parametric memory — auditable only by hand.
- **Lec10** assembles full research cases from these parts.

$$\underbrace{\text{LLM}}_{\text{language engine (Lec07)}} + \underbrace{\text{retrieval}}_{\text{grounded memory (Lec08)}} + \underbrace{\text{tools}}_{\text{grounded action (Lec09)}} \Rightarrow \text{auditable economic measurement}$$

Arc complete: T1 FRAME → T2 REPRESENT → T3 ATTEND → T4 PRETRAIN → **T5 MEASURE.**